

HOW TO USE

이 자료 사용 방법

1. 수업·강의·자습에 그대로 쓰셔도 됩니다. 출처만 남겨 주세요.
2. PDF 는 화면으로 읽는 용도, PPTX 는 발표용입니다. PPTX 는 Pretendard 글꼴이 설치된 컴퓨터에서 가장 똑같이 보입니다.
3. 출처 표기 예: 이애본(드림아이티비즈), 「자료 제목」, 홍문원, useful-ai.dreamitbiz.com, CC BY-NC-ND 4.0
4. 자료 안의 프롬프트는 복사해 바로 쓸 수 있습니다. [대괄호]만 내 상황으로 바꾸세요.
5. 모든 자료와 영상 강의는 useful-ai.dreamitbiz.com 에서 다시 받을 수 있습니다.

저작권 · 이용 조건 (CC BY-NC-ND 4.0)

1. 저작권 © 2026 이애본(드림아이티비즈 DreamIT Biz). 발행 홍문원(洪文院, 출판사 신고번호 제2026-000026호). 이애본이 2026년 대학·기관 강의 현장에서 쓴 강의안을 전자출판용으로 다시 묶은 것입니다. ISBN: 등록 후 기재.
2. 이 자료는 크리에이티브 커먼즈 저작자표시-비영리-변경금지 4.0 국제(CC BY-NC-ND 4.0) 라이선스를 따릅니다.
creativecommons.org/licenses/by-nc-nd/4.0/deed.ko
3. 저작자 표시(BY): 출처를 밝히면 그대로 복사해 수업·강의·자습에 쓸 수 있습니다. 비영리(NC): 유료 강의·판매·광고 수익 목적에는 쓸 수 없습니다. 변경금지(ND): 내용을 고치거나 잘라 재조합해 배포할 수 없습니다.
4. 자료 안의 외부 서비스 화면 캡처와 상표는 교육 목적 인용이며 각 서비스의 소유입니다. AI 생성 이미지는 생성 도구의 이용 약관을 따릅니다.
5. 라이선스 범위 밖의 이용(영리·변형·2차 저작)은 먼저 문의해 주세요. aebon@dreamitbiz.com · 카카오톡 ID aebon

회사 임직원을 위한 풀스택 AI 개발 워크숍 · 3일 · 21시간

Claude Code와 함께하는 풀스택 바이브코딩 실무 워크숍

대상: 회사 임직원(비개발자 포함) · 방식: 대면 실습형 · 총 시간: 21시간(3일)

자연어로 요구하면 코드가 만들어지는 시대 — Cursor IDE · React · Supabase · Streamlit

\$ vibe-coding --start

▶ Day 1 · Frontend

Cursor IDE · React · TypeScript

▶ Day 2 · Fullstack

Express · Supabase · JWT/OAuth

▶ Day 3 · Data & AI

Streamlit · pandas · Claude API

Ph.D. 이애본 (aebon@dreamitbiz.com) / 드림아이티비즈(DreamIT Biz)

워크숍 개요와 준비사항

3일간 무엇을, 어떻게, 어떤 도구로 만들 것인지 한 페이지로 확인합니다.

TOTAL HOURS

21 시간

3일 · 8h + 8h + 5h

PROJECTS

24 선

웹 · Streamlit · 데이터 · AI 4분야

PROD-READY APPS

2 개

풀스택 웹 + Streamlit Cloud

CORE TOOL

Cursor IDE
+ Claude Code

AI 네이티브 코드 에디터

| 사전 준비물 체크리스트

노트북— Windows / macOS 모두 가능, 8GB RAM 이상 권장

GitHub 계정— github.com에서 가입 (무료)

이메일 계정— Supabase / Anthropic / Cursor 가입용

안정적 인터넷— AI API 호출, 패키지 다운로드

학습 태도— "왜 안 되지?"보다 AI에게 물어보기

자연어 설명력— 코드보다 의도(intent) 전달이 중요

페어 학습— 막힐 땐 옆자리, 그다음 AI, 그다음 공식 문서

완성보다 도전— 1개라도 끝까지 배포해 보기



LEARNING GOALS

3일 뒤, 당신은 이 4가지를 할 수 있습니다

자연어로 의도를 설명하면
AI가 코드를 생성하는
새로운 개발 방식으로
실제 동작하는 서비스를 만듭니다.

01

바이브코딩 패러다임을 체득한다

Cursor IDE의 Cmd+K, Cmd+L, Agent를 손에 익히고, AI에게 "무엇을 만들지" 명확하게 전달하는 프롬프트 감각을 갖춥니다.

02

React 기반 프론트엔드를 만든다

컴포넌트·Props·State·Router 개념을 이해하고, AI에게 요구사항을 전달해 화면 단위로 UI를 조립할 수 있습니다.

03

DB와 인증까지 풀스택을 연결한다

REST API · Supabase(PostgreSQL) · JWT/OAuth로 회원가입·로그인이 동작하는 진짜 서비스를 완성합니다.

04

데이터 앱 + AI 챗봇을 배포한다

Streamlit으로 전력 데이터 대시보드를 만들고 Claude API를 연동해 Streamlit Cloud에 무료 배포까지 마칩니다.

중요한 것은 "AI가 코드를 쓰게 하는 것"이 아니라
"AI와 함께 더 빠르게, 더 잘 동작하는 결과물을 만드는 것"입니다.

3일, 21시간 — 한 페이지 커리큘럼

TOTAL
21H · 3일

DAY 1 · 8H

바이브코딩 + 프론트엔드

09:00 — 12:00 · 3H

Part 1. 바이브코딩 & 개발 환경

Cursor IDE · Node.js · Git · GitHub

13:00 — 15:00 · 2H

Part 2. 서비스 기획 & 요구사항

Figma · AI 기획서 · 폴더 구조

15:00 — 16:00 · 1H

Part 3. HTML/CSS/JS 기초

Flexbox · TypeScript · 핵심 문법

16:00 — 18:00 · 2H

Part 4. React UI 구현 실습

JSX · Props · State · Router

DAY 2 · 8H

백엔드 + 풀스택 통합

09:00 — 12:00 · 3H

Part 1. REST API & Node.js

Express · CRUD · 미들웨어 · Postman

13:00 — 16:00 · 3H

Part 2. DB & 프론트-백 통합

Supabase · PostgreSQL · RLS

16:00 — 18:00 · 2H

Part 3. 인증·인가 (JWT/OAuth)

로그인·회원가입 · Google/Kakao

종합 실습

Day 1 프론트엔드와 연결하여
실제 동작하는 풀스택 앱 완성

DAY 3 · 5H

Streamlit · 데이터 · AI 배포

09:00 — 11:00 · 2H

Part 1. Streamlit + Python

설치 · 위젯 · 자기소개 앱

11:00 — 12:00 · 1H

Part 2. 데이터 앱 개발

pandas · 차트 · 전력 대시보드

13:00 — 15:00 · 2H

Part 3. AI 연동 + Cloud 배포

Claude API · Streamlit Cloud

FINALE

최종 프로젝트 발표
+ 상호 피드백

DAY ONE · 8 HOURS

01

바이브코딩 입문 & React 프론트엔드

오늘 끝에는 — AI에게 자연어로 요청해 React Todo 앱을 만들 수 있습니다.

TODAY'S SCHEDULE

09:00 — 12:00

Part 1. 바이브코딩 개요 & 개발 환경 구성

3H · Cursor · Node · Git

13:00 — 15:00

Part 2. 서비스 기획 & 요구사항 정의

2H · Figma · AI 기획서

15:00 — 16:00

Part 3. HTML/CSS/JS 기초

1H · TypeScript

16:00 — 18:00

Part 4. React UI 구현

2H · 실습

DAY 1 · PART 1

01

/ 04 parts

09:00 — 12:00

3시간 · 도입 세션

PART 1.

바이브코딩 개요 & 개발 환경 구성

LEARNING OUTCOMES

바이브코딩의 정의와 작동 원리 이해

Cursor IDE 설치 및 핵심 단축키 숙지

Node.js · Git · GitHub 환경 구성

Vite + React + TS 프로젝트 생성

바이브코딩(Vibe Coding)이란?

AI에게 자연어로 의도(vibe)를 전달하면, AI가 코드를 생성하는 새로운 개발 방식.

BEFORE · 기존 코딩

```
// 1. 문법을 외운다
const [todos, setTodos] = useState([])
// 2. 한 줄 한 줄 직접 타이핑
const add = (t) => setTodos([...todos, t])
// 3. 에러 메시지 검색 → StackOverflow
// 4. 동작할 때까지 디버깅 반복
```

문법·디버깅에 시간 대부분

AFTER · 바이브코딩

```
$ vibe-coding
```

"할 일 관리 앱 만들어줘.
입력창, 체크박스, 삭제 버튼.
완료된 건 취소선으로.
React + TypeScript로."

→ AI generates working code

의도 설명·검토에 시간 대부분

바이브코딩의 3가지 핵심 변화

FROM "HOW" TO "WHAT"

문법 → 의도

"어떻게 짜지"가 아니라 "무엇을 만들지"에 집중합니다.

FAST PROTOTYPE

빠른 시제품

며칠 걸리던 시제품을 훨씬 짧은 시간에 만들어 봅니다.

REVIEWER, NOT TYPIST

개발자 = 리뷰어

코드를 "쓰는 사람"에서 "지시하고 검증하는 사람"으로.

AI Agent와 코딩 어시스턴트

두 가지를 구분하면 어떤 도구를 언제 써야 하는지 명확해집니다.

CODING ASSISTANT

코딩 어시스턴트

당신이 코드를 쓰고, AI가 옆에서 돕는 모드.
자동 완성, 에러 수정, 리팩토링, 한 줄 설명을 보조합니다.

AI AGENT

AI 에이전트

목표를 주면 AI가 스스로 일합니다.
계획 → 파일 생성 → 코드 작성 → 테스트 실행까지 자율적으로 수행.

대표적인 4가지 도구



Cursor IDE

RECOMMENDED · 본 강의

AI 네이티브 코드 에디터.
GPT·Claude·Gemini 등 모델 선택.
Agent 모드로 여러 파일을 한 번에 편집.



GitHub Copilot

VS Code 확장

VS Code에 붙여 쓰는 보조 도구.
자동완성 강점.
기업 환경에서 널리 쓰임.



Claude Code

TERMINAL · 본 강의

터미널·IDE에서 동작하는 AI 코딩 Agent.
긴 컨텍스트, 멀티파일,
자율 실행에 강합니다.



Windsurf

PAIR PROGRAMMING

AI 페어 프로그래밍 IDE.
에이전트(Cascade)와 함께 짜는 IDE.

개발 환경 4종 세트

이 네 가지가 깔려 있어야 바이브코딩이 시작됩니다. 하나씩 같이 설치합니다.

01 · RUNTIME



Node.js

v24 LTS

JavaScript 런타임.
React 프로젝트 생성, npm 패키지 설치, 개발
서버 실행에 필수입니다.

```
$ node -v  
v24.21.0
```

02 · CORE TOOL



Cursor IDE

Latest

AI 네이티브 코드 에디터.
GPT·Claude 등 모델 선택.
바이브코딩의 본진입니다.

```
→ cursor.com  
Download & Sign in
```

03 · VERSION CONTROL



Git

Latest

버전 관리 시스템.
코드 변경 이력 추적, 브랜치 분기, 되돌리기가
가능합니다.

```
$ git --version  
git version 2.43.0
```

04 · REMOTE



GitHub

Free 계정

코드 원격 저장소 + 무료 배포.
GitHub Pages로 정적 사이트를 무료로
호스팅할 수 있습니다.

```
→ github.com  
Sign up · 무료
```

Cursor IDE 핵심 단축키 4개

이 네 가지 단축키만 손에 익으면, 오늘부터 바이브코딩이 가능합니다.

INLINE GENERATE

⌘ K (Windows: Ctrl+K)

인라인 코드 생성·수정

코드 위에서 누르면 → 자연어로 변경 요청. 선택한 코드 블록만 정확히 바꿔줍니다.

CHAT PANEL

⌘ L (Windows: Ctrl+L)

AI 채팅 패널 열기

오른쪽 사이트에 챗 패널이 열립니다. 코드 외 일반 질문이나 설계 상의에 사용.

AGENT MODE

⌘ I (Agent)

Agent · 멀티파일 자율 편집

"Todo 앱 만들어줘"처럼 큰 작업을 던지면 여러 파일을 한 번에 만들거나 수정합니다.

CONTEXT REFERENCE

@파일명 in chat

특정 파일 참조 대화

"@App.tsx 이 컴포넌트를 분리해줘"처럼 정확한 파일을 콕 집어 컨텍스트를 제공합니다.

실무 팁 · 큰 작업은 Agent(⌘I), 한 줄 수정은 인라인(⌘K), 의논은 채팅(⌘L) — 이 3개로 끝납니다.

npm·Git 핵심 명령어 + 첫 프로젝트 실습

외우지 마세요. 이 페이지를 옆에 두고 하나씩 따라쳐 보면 됩니다.

Step 1 · Vite + React + TS 프로젝트 생성

```
# 버전 확인
$ node -v && npm -v

# 새 프로젝트 생성 (Vite + React + TypeScript)
$ npm create vite@latest my-app -- --template react-ts

# 의존성 설치 + 개발 서버 실행
$ cd my-app && npm install
$ npm run dev

→ http://localhost:5173
```

Step 2 · Git으로 첫 커밋 → GitHub 푸시

```
# 저장소 초기화 + 첫 커밋
$ git init -b main
$ git add .
$ git commit -m "첫 번째 커밋"

# GitHub 원격 연결 + 푸시
$ git remote add origin <repo-url>
$ git push -u origin main

→ GitHub에 코드 업로드 완료
```

실습 · HANDS-ON

Cursor IDE로 첫 React 프로젝트 만들고 카운터 앱 생성하기

- ① Vite + React + TS 프로젝트 생성 → 개발 서버 실행
- ② Cursor에서 ㉸ → "간단한 카운터 앱 만들어줘"

- ③ 동작 확인 후 git init → 첫 커밋
- ④ GitHub에 새 리포 만들고 푸시

DAY 1 · PART 2

02

/ 04 parts

13:00 — 15:00

2시간 · 기획 세션

PART 2.

웹 서비스 기획 & 요구사항 정의

CORE QUESTION

"코드를 짜기 전에
무엇을 만들지 정하는 것"
이 진짜 시작입니다.

기획서 · 와이어프레임 · 폴더 구조를 AI와 함께 한 시간 안에 완성합니다.

웹 서비스 기획 5단계

코드는 AI가 짜더라도 — "무엇을 만들지" 결정은 사람이 합니다. 5단계로 정리합니다.



BIG IDEA · 본 강의의 접근법

AI에게 기획부터 시킵니다.

아이디어 한 줄을 던지면 AI가 → 페르소나·기능 목록·화면 흐름·DB 스키마·API 엔드포인트까지 한 번에 초안을 만듭니다.
우리는 그것을 검토하고 다듬는 역할입니다.

AI에게 요구사항 정의서 받는 프롬프트

"기획서 좀 짜줘" — 이런 두루뭉술한 요청 대신, 5요소를 채워 보냅니다.

Cursor Chat · 기획 프롬프트 템플릿

너는 웹 서비스 기획 전문가야.

다음 조건의 요구사항 정의서를 작성해줘.

[조건]

- 서비스: 할 일 관리(Todo) 웹앱
- 사용자: 개인 직장인
- 핵심 기능: CRUD + 완료체크 + 카테고리
- 기술 스택: React + Supabase
- 인증: 이메일 로그인 + Google OAuth

[출력 형식]

1. 프로젝트 개요 (목적·범위)
2. 기능 목록 (필수/선택 구분)
3. 화면 목록 (페이지 명세)
4. 데이터 모델 (테이블 설계)
5. API 엔드포인트 목록

마크다운 표 형식으로 출력해줘.

좋은 기획 프롬프트의 5요소

1

Role · 역할

"너는 ○○ 전문가야" — 답변 톤이 결정됩니다.

2

Objective · 목적

"무엇을 만들지" — 결과물의 형태를 적습니다.

3

Context · 맥락

사용자·도메인·제약 등 배경 정보 전달.

4

Format · 형식

표·리스트·코드 블록 등 출력 구조 지정.

5

Constraint · 조건

길이·어조·반드시 포함/제외할 사항.

핵심 팁

처음엔 두루뭉술해도 OK — AI에게 추가 질문을 받으며 점점 구체화하세요.

화면 설계 + 폴더 구조

기획서가 끝나면 → Figma로 화면을 그리고 → 폴더 구조까지 한 번에 만듭니다.

FIGMA · 화면 설계

와이어프레임부터 그립니다



Login

Todo List

My Page

Tip 1. 픽셀 단위 디자인은 나중에. 처음엔 박스와 선만 그립니다.

Tip 2. Cursor에서 Figma 화면을 스크린샷으로 첨부 → AI가 코드로 변환.

Tip 3. 화면 흐름(이동 경로)을 화살표로 표시하면 라우터 설계가 쉬워집니다.

FOLDER STRUCTURE · React 표준

my-app/ 디렉토리

```
my-app/  
├─ public/  // 정적 파일 (이미지·아이콘)  
├─ src/  
│   ├─ components/  // 재사용 UI  
│   ├─ pages/  // 페이지 단위  
│   ├─ contexts/  // 전역 상태  
│   ├─ utils/  // 헬퍼 함수  
│   ├─ types/  // 타입 정의  
│   ├─ App.tsx  // 라우터 설정  
│   └─ main.tsx  // 엔트리  
├─ package.json  
├─ tsconfig.json  
└─ vite.config.ts
```

DAY 1 · PART 3

03

/ 04 parts

15:00 — 16:00

1시간 · 압축 세션

PART 3.

프론트엔드 기초 HTML · CSS · JS · TS

PHILOSOPHY

"모든 문법을 외우지 않아도 됩니다."

개념 · 구조 · 핵심 키워드만 이해하면 됩니다.
나머지는 AI가 정확하게 채워줄 수 있습니다.

비개발자도 1시간이면 핵심을 잡을 수 있도록 설계되었습니다.

HTML 시맨틱 + CSS Flexbox / Grid

3가지만 기억하세요 — 의미·정렬·반응형.

HTML · 시맨틱 태그가 핵심

```
<!DOCTYPE html>
<html lang="ko">
  <head>...</head>
  <body>
    <header> // 페이지 상단
      <h1>할 일 관리</h1>
      <nav>네비</nav>
    </header>
    <main> // 메인 콘텐츠
      <section>...</section>
      <ul><li>...</li></ul>
    </main>
  </body>
```

핵심 시맨틱 태그

header · nav · main · section · article · aside · footer · ul/li · button

→ <div> 대신 의미 있는 태그를 — SEO와 접근성에 유리합니다.

CSS · 핵심 3가지

① Flexbox · 1차원 정렬

```
.header { display:flex; justify-content:space-between;
align-items:center; }
```

→ 좌·우 자동 분리. 가장 자주 쓰는 레이아웃.

② Grid · 2차원 카드 레이아웃

```
.cards { display:grid;
grid-template-columns:repeat(3, 1fr); gap:16px; }
```

→ 카드를 3열로 정렬. 대시보드 단골.

③ @media · 반응형

```
@media (max-width: 768px) {
  .cards { grid-template-columns: 1fr; }
}
```

→ 모바일에선 1열로 자동 전환.

JavaScript · TypeScript 핵심

React를 만지기 전에, 이 6가지만 직관적으로 이해하면 충분합니다.

01 · 변수 선언

const & let

```
const name = "홍길동"; // 상수
let count = 0; // 변경 가능
```

02 · 함수

화살표 함수

```
const add = (a, b) => a + b;
add(1, 2) // 3
```

03 · 배열 메서드

map · filter

```
arr.map(x => x * 2)
arr.filter(x => x > 0)
```

04 · 객체 다루기

구조분해 + 스프레드

```
const {name, age} = user;
const arr2 = [...arr, item];
```

05 · 비동기

async / await

```
const res = await fetch(url);
// API 호출에 자주 사용
```

06 · 타입

TypeScript interface

```
interface Todo {
  id: number; title: string;
}
```

바이트코딩 팁

"문법은 외우지 마세요. 개념만 이해하고 AI에게 정확히 요청하세요."

중요한 것은 "무엇을 만들고 싶은지"를 명확하게 전달하는 것. 세부 문법은 AI가 대신 작성합니다.

DAY 1 · PART 4 · FINALE

04

/ 04 parts

16:00 — 18:00

2시간 · 첫 결과물

PART 4.

React 기반 프론트엔드 UI 구현

TODAY'S DELIVERABLE

동작하는 Todo 앱

컴포넌트 · Props · State · Router · 자동 컴포넌트 생성까지
— Day 1의 모든 도구를 한 화면 안에서 통합합니다.

오늘 끝에는 — 친구에게 URL 공유 가능한 완성된 앱이 나옵니다.

React 핵심 5개념

이 5가지 단어만 정확히 이해하면 — React 화면을 AI에게 요청할 수 있습니다.

CONCEPT 01 function block

Component

UI의 재사용 가능한 조각.
레고 블록 같은 것.

```
function TodoItem() {  
  return <li>...</li>;  
}
```

CONCEPT 02 JS + HTML

JSX

JavaScript 안에
HTML을 쓰는 문법.

```
const el =  
<h1>Hello {name}</h1>;  
// 변수 → 중괄호로
```

CONCEPT 03 parent → child

Props

부모 → 자식 컴포넌트로
데이터를 내려주는 통로.

```
<TodoItem  
  title="공부"  
  done={true} />
```

CONCEPT 04 internal data

State

컴포넌트 내부의
변경 가능한 데이터.

```
const [n, setN] =  
  useState(0);  
setN(n + 1) // 변경
```

CONCEPT 05 side effects

Effect

렌더 후 부가 작업
(API · 타이머 등).

```
useEffect(() => {  
  fetchData();  
}, []);
```

DATA FLOW · 한 줄 요약

State 변경 → 컴포넌트 다시 렌더 → Props로 자식에게 새 값 전달

데이터는 위에서 아래로만 흐릅니다. 자식이 부모를 바꾸려면 → 부모가 전달한 함수(콜백)를 호출합니다.

Component + useState 실전

Todo 앱의 핵심 두 가지 — 한 줄로 새 할 일 추가하고 체크하는 로직.

TodoItem.tsx · 자식 컴포넌트

```
// 1) Props 타입을 먼저 정의 — TS의 핵심
interface Props {
  todo: { id: number; title: string; done: boolean };
  onToggle: (id: number) => void;
}

// 2) Props를 구조분해로 받기
const TodoItem = ({ todo, onToggle }: Props) => (
  <li className={todo.done ? 'done' : ''}>
    <input type="checkbox"
      checked={todo.done}
      onChange={() => onToggle(todo.id)} />
    <span>{todo.title}</span>
  </li>
);
```

App.tsx · 부모 컴포넌트 (useState)

```
// 1) State 선언 (배열)
const [todos, setTodos] = useState<Todo[]>([]);
const [input, setInput] = useState('');

// 2) 새 할 일 추가
const addTodo = () => {
  if (!input.trim()) return;
  setTodos(prev => [...prev,
    { id: Date.now(), title: input, done: false }
  ]);
  setInput('');
};

// 3) 완료 토글 (map으로 불변 업데이트)
const toggle = (id: number) =>
  setTodos(prev => prev.map(t =>
    t.id === id ? { ...t, done: !t.done } : t));
```

Router + AI에게 UI 만들어달라기 + 실습

페이지를 분리하고, AI에게 정확하게 컴포넌트를 만들어 달라고 요청하는 패턴.

React Router · 페이지 분리

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';  
<BrowserRouter>  
  <Routes>  
    <Route path="/" element={<Home/>} />  
    <Route path="/todos" element={<TodoApp/>} />  
  </Routes>  
</BrowserRouter>
```

Cursor · AI (Agent) 프롬프트 예시

"할 일 관리 앱의 메인 페이지를 만들어줘.
상단에 **입력 폼**, 하단에 **할 일 목록**이 표시되고,
각 항목엔 **체크박스**와 **삭제 버튼**이 있어.
완료된 항목은 **취소선**으로 표시.
깔끔하고 모던한 **CSS**도 함께 작성해줘.
React + TypeScript로, **useState** 사용."

실습 · 1일차 최종 결과물

AI에게 요청해서 Todo 앱 완성하기 (2시간)

1

Agent 요청

⌘I → 위 프롬프트 그대로 붙여넣고 Enter.

2

코드 검토 · Accept

생성된 파일을 한 번 훑고 Accept all.

3

실행 · 동작 확인

npm run dev → 브라우저에서 동작 점검.

4

개선 요청

"카테고리 필터 추가해줘" 등 반복.

APPENDIX · A

PYTHON ESSENTIALS

Python

필수 문법 10선

Streamlit · 데이터 분석 · AI API 연동에 꼭 필요한
핵심 문법만 골라서 — 10장으로 정리.

`variables · strings · list · dict · if · for · def · comprehension · import ·
try/except`

변수와 자료형 — 모든 데이터의 출발점

타입을 선언하지 않아도 됩니다. 값을 넣으면 Python이 알아서 판단합니다.

variables.py

```
# ① 숫자 — 정수, 실수
age = 28 # int
price = 9.5 # float

# ② 문자열 — 따옴표면 OK
name = "홍길동" # str

# ③ 참/거짓
is_admin = True # bool

# ④ 값이 없음 — None
result = None # NoneType

# ⑤ 타입 확인 · 변환
print(type(age)) # <class 'int'>
n = int("42") # 문자열 → 숫자
```

POINT 01 · 타입 자동 추론

값을 대입하는 순간 타입이 정해진다.
JavaScript처럼 let / const 키워드 없음.

POINT 02 · 변수명 규칙

snake_case 사용. 숫자로 시작 **x**,
예약어(if · for · class) 사용 **x**

POINT 03 · f-string

```
name = "길동"
print(f"안녕, {name}님")
```

현업에서 자주 쓰는 패턴

설정값은 대문자 API_KEY · 임시 변수는 _ · 비공개는 앞에
_name

문자열 다루기 — 핵심 메서드 6개

사용자 입력·CSV·API 응답은 모두 문자열에서 시작합니다.

.upper() · .lower()

대문자·소문자 변환. **검색·비교 전 정규화**에 자주 사용.

```
"DATA".lower()  
# → "data"
```

.strip()

앞뒤 **공백·줄바꿈 제거**. 사용자 입력 정리에 필수.

```
" hello ".strip()  
# → "hello"
```

.split() · .join()

구분자로 **자르기/합치기**. CSV·URL 파싱 단골.

```
"a,b,c".split(",")  
# → ['a', 'b', 'c']
```

.replace()

부분 문자열 **치환**. 두 번째 인자는 새 문자열.

```
"010-1234".replace("-", "")  
# → "0101234"
```

in · 포함 검사

부분 문자열 **포함 여부**를 True/False로 반환.

```
"py" in "python"  
# → True
```

f-string

변수 삽입 + **서식**까지. 가장 자주 쓰는 출력 방식.

```
f"가격: {price:,.0f}원"  
# → "가격: 9,500원"
```

리스트 · 튜플 — 순서 있는 묶음

데이터가 여러 개일 때 가장 먼저 떠올리는 자료형입니다.

list — 수정 가능한 묶음 []

```
# 생성 — 대괄호
teams = ["개발", "운영", "기획"]

# 인덱싱 — 0부터, 음수는 뒤에서
teams[0] # "개발"
teams[-1] # "기획"

# 슬라이싱 [시작:끝]
teams[0:2] # ['개발', '운영']

# 추가 · 삭제
teams.append("디자인")
teams.remove("운영")

# 길이 · 정렬
len(teams) # 3
sorted(teams) # 새 정렬 리스트
```

tuple — 수정 불가 ()

```
# 좌표 · RGB · 고정 설정에 사용
point = (10, 20)
rgb = (26, 54, 93)
x, y = point # 언패킹
```

언제 무엇을 쓸까?

list []

- ✓ 동적 데이터
- ✓ 추가·삭제 빈번
- ✓ 사용자가 선택한 항목들
- ✓ 대부분의 경우

tuple ()

- ✓ 좌표 · 색상
- ✓ 함수가 여러 값 반환
- ✓ dict의 키로 사용
- ✓ 변하지 않을 값

dict · set — 키로 찾고, 중복 없이 모으기

JSON 응답·DB 행·설정값을 다룰 때 가장 많이 등장합니다.

dict — 키:값 쌍 { }

```
# 생성
user = {
    "name": "홍길동",
    "age": 28,
    "team": "개발"
}

# 값 읽기 · 수정 · 추가
user["name"] # "홍길동"
user["age"] = 29 # 수정
user["email"] = "a@example.com" # 추가

# 안전한 조회 (없으면 기본값)
user.get("phone", "미입력")

# 키 · 값 · 항목 순회
for k, v in user.items():
    print(k, v)
```

set — 중복 없는 묶음

```
# 중복 자동 제거
tags = {"AI", "DB", "AI"}
# → {"AI", "DB"}

# 리스트의 중복 제거
unique = list(set(arr))

# 멤버십 검사 (매우 빠름)
"AI" in tags # True

# 집합 연산
a | b # 합집합
a & b # 교집합
a - b # 차집합
```

조건문 — if · elif · else

중괄호 없이 **들여쓰기 4칸**으로 블록을 구분합니다.

grade.py

```
score = 85

if score ≥ 90:
    grade = "A"
elif score ≥ 80:
    grade = "B"
elif score ≥ 70:
    grade = "C"
else:
    grade = "F"

# 한 줄 if — 삼항 연산자
msg = "합격" if score ≥ 60 else "불합격"
```

비교 연산자

== 같다 != 다르다

> ≥ < ≤

in 포함 not in 미포함

논리 연산자

and 둘 다 참

or 둘 중 하나라도 참

not 반대

들여쓰기 = 문법

스페이스 4칸으로 통일.

탭과 스페이스를 섞으면 IndentationError 발생.

반복문 — for · while

Python의 for는 "꺼내서 처리"입니다. 인덱스가 아니라 값을 직접 받습니다.

for — 컬렉션을 순회

```
# ① 리스트 순회
for name in ["길동", "영희"]:
    print(name)

# ② 숫자 범위 — range()
for i in range(5): # 0,1,2,3,4
    print(i)

# ③ 인덱스가 필요할 때 — enumerate
for idx, name in enumerate(arr):
    print(f"{idx}: {name}")

# ④ dict 순회
for k, v in user.items():
    print(k, v)
```

while — 조건이 참인 동안

```
count = 0
while count < 3:
    print(count)
    count += 1 # ★ 꼭 증가시켜야 함
```

흐름 제어 — break · continue

```
for n in range(10):
    if n == 5:
        break # 반복 즉시 종료
    if n % 2 == 0:
        continue # 다음 회차로 점프
    print(n)
```

함수 정의 — def · 인자 · return

반복되는 코드는 함수로. "이름 붙이기 = 추상화의 시작"입니다.

functions.py

① 기본 형태

```
def greet(name):
    return f"안녕, {name}님"
greet("길동") # → "안녕, 길동님"
```

② 기본값 — 인자 생략 가능

```
def power(x, n=2):
    return x ** n
power(3) # 9 (n=2 사용)
power(3, 3) # 27
```

③ 여러 값 반환 — 튜플

```
def stats(nums):
    return min(nums), max(nums), sum(nums)
lo, hi, total = stats([3, 7, 1])
```

④ 타입 힌트 (선택, 추천)

```
def add(a: int, b: int) → int:
    return a + b
```

함수 4구성 요소

- ① def 키워드 ② 함수 이름
- ③ 인자(괄호 안) ④ return 반환값

키워드 인자

이름으로 전달 — 순서 무관
power(n=3, x=2)

함수 = 단일 책임

함수 이름을 **동사로** 짓고,
한 함수는 **한 가지만** 하도록. AI도 이런 함수를 더 잘 만듭니다.

컴프리헨션 — 한 줄로 끝내는 반복

"리스트를 만들기 위해 빈 리스트 + for + append" 패턴을 **한 줄로**.

BEFORE — for문 3줄

```
squares = []
for n in range(5):
    squares.append(n ** 2)

# → [0, 1, 4, 9, 16]
```

AFTER — 컴프리헨션 1줄

```
squares = [n**2 for n in range(5)]

# → [0, 1, 4, 9, 16]

[ 표현식 for 변수 in 컬렉션 ]
```

+ 조건 필터

```
# 짝수만
[n for n in nums
 if n % 2 == 0]

# → [2, 4, 6]
```

문자열 가공

```
# 모두 대문자로
[n.upper()
 for n in names]

# ["KIM", "LEE", "PARK"]
```

dict comprehension

```
# {키: 값} 만들기
{n: n**2
 for n in range(4)}

# {0:0, 1:1, 2:4, 3:9}
```

import · pip — 남의 코드 가져 쓰기

Python의 진짜 강점은 **방대한 라이브러리**. pip로 설치하고 import로 씁니다.

terminal

```
$ pip install streamlit
$ pip install pandas plotly
$ pip install anthropic

# 설치된 패키지 목록
$ pip list
$ pip freeze > requirements.txt
```

import 4가지 패턴

```
import pandas
# pandas.DataFrame(...) — 전체 사용

import pandas as pd
# pd.DataFrame(...) — 별명 (관례)

from pandas import DataFrame
# DataFrame(...) — 특정 것만

from datetime import datetime, timedelta
```

이번 과정에서 꼭 알아야 할 6개

streamlit 웹 UI 프레임워크

```
import streamlit as st
```

pandas 표 형태 데이터 처리

```
import pandas as pd
```

plotly 인터랙티브 차트

```
import plotly.express as px
```

anthropic Claude API 클라이언트

```
from anthropic import Anthropic
```

requests HTTP 요청

```
import requests
```

openpyxl Excel 파일 읽기/쓰기

```
import openpyxl
```

예외 처리 — try · except · finally

에러를 만나도 **프로그램이 안 죽도록** — API · 파일 · 사용자 입력에 필수.

```
api_call.py

# Claude API 호출 — 실패할 수 있음
try:
    response = client.messages.create(...)
    answer = response.content[0].text

except APIConnectionError:
    answer = "네트워크 연결 실패"

except RateLimitError:
    answer = "요청 한도 초과 — 잠시 후 재시도"

except Exception as e:
    print(f"예상 못한 에러: {e}")
    answer = None

finally:
    print("요청 종료 — 로그 기록")

# try → 시도 / except → 실패 시 / finally → 무조건
```

자주 만나는 예외

ValueError 형변환 실패
 KeyError dict 키 없음
 FileNotFoundError 파일 없음
 ZeroDivisionError 0으로 나눔

에러 일으키기 — raise

```
if age < 0:
    raise ValueError("음수 불가")
```

실무 규칙

- ① 예상되는 예외만 잡기
- ② 광범위한 except: 단독은 금지
- ③ except 안에서 **로그 + 대체값**

Streamlit

필수 문법 10선

Python 한 파일로 — 데이터 앱이 웹이 됩니다.
st. 한 글자로 모든 위젯을 호출하세요.

`run · text · input · select · button · data · chart · layout · session_state`

설치 → 실행까지 60초

HTML도 CSS도 React도 없이 — Python 파일 하나가 곧 웹 앱입니다.

STEP 1 · 설치

```
$ pip install streamlit

# 가상환경 권장 (Mac/Linux):
$ python -m venv venv
$ source venv/bin/activate
$ pip install streamlit
```

STEP 2 · app.py 작성

```
import streamlit as st

st.title("안녕, Streamlit!")
st.write("첫 번째 데이터 앱")

name = st.text_input("이름?")
st.write(f"환영합니다, {name}님")
```

STEP 3 · 실행 ★

```
$ streamlit run app.py

# 자동으로 브라우저 열림
Local URL: http://localhost:8501
Network URL: http://192.168.x.x:8501

# 파일 저장 시 자동 새로고침 ✓
```

Streamlit의 정신모델

- ① 위에서 아래로 **한 번 실행**하면 그 결과가 그대로 화면이 된다.
- ② 사용자가 위젯을 만지면 파일 전체를 다시 실행한다.
- ③ "이벤트 핸들러"라는 개념이 없다 — 그래서 React보다 훨씬 쉽다.

텍스트 출력 — 6가지 형태

제목 · 본문 · 코드 · 메시지를 한 줄로 출력합니다.

text_demo.py

```
# ① 제목 — 페이지 최상단
st.title("전력 사용량 대시보드")

# ② 섹션 헤더
st.header("1. 월별 추이")
st.subheader("광주지역")

# ③ 만능 출력 — write
st.write("문자열도 OK")
st.write(df) # DataFrame
st.write({"a": 1}) # dict

# ④ 마크다운 — 가장 자주 사용
st.markdown("**꺾게** 와 *기울임*")

# ⑤ 코드 블록
st.code("print('hi')", language="python")

# ⑥ 상태 메시지 (4종)
st.success("저장 완료") st.info("안내")
st.warning("주의") st.error("실패")
```

localhost:8501

전력 사용량 대시보드

1. 월별 추이

광주지역

꺾게 와 기울임

```
print('hi')
```

저장 완료

안내

주의

실패

입력 위젯 — 사용자가 값을 넣는 곳

반환값을 변수에 받아서 곧바로 사용 — 그게 전부입니다.

st.text_input() — 한 줄 문자

```
name = st.text_input(
    "이름", value="", placeholder="홍길동"
)
```

홍길동

st.number_input() — 숫자

```
age = st.number_input(
    "나이", min_value=0, max_value=120, step=1
)
```

28

+

st.text_area() — 여러 줄 문자

```
memo = st.text_area("메모", height=100)
```

오늘 회의 핵심:
1) Vibe coding 도입
2) Day 2까지 API 완성

st.slider() — 범위 선택

```
temp = st.slider("온도", 0, 100, 36)
```

36

0 ~ 100

선택 위젯 — 옵션 중에서 고르기

필터·옵션·체크박스 — 4가지 위젯으로 모든 선택 UI 처리.

st.selectbox() — 하나 선택 (드롭다운)

```
region = st.selectbox(
    "지역", ["광주", "전남", "서울"]
)
```

광주 ▼

st.multiselect() — 여러 개 선택

```
tags = st.multiselect(
    "태그", ["AI", "DB", "보안"], default=["AI"]
)
```

AI ×

DB ×

st.radio() — 라디오 버튼

```
plan = st.radio("요금제", ["무료", "Pro", "기업"], horizontal=True)
```



무료



Pro



기업

st.checkbox() · st.toggle()

```
show = st.checkbox("고급 옵션 표시")
```

```
dark = st.toggle("다크 모드")
```



고급 옵션 표시



다크 모드

버튼과 폼 — 사용자 액션 받기

`button = True/False` · `form = 입력 묶음 + 제출 버튼` — 차이를 이해하면 끝.

st.button() — 클릭 한 번

```
# 클릭되면 True 반환
if st.button("실행"):
    st.success("분석 완료!")

# 스타일 강조 — type="primary"
st.button("저장", type="primary")

# 다운로드 버튼 — 파일 받기
st.download_button(
    "CSV 다운로드",
    data=df.to_csv(),
    file_name="result.csv"
)

# button은 클릭 직후 한 번만 True
# 상태 유지가 필요하면 session_state 사용
```

st.form() — 모았다가 한 번에 제출

```
# with 블록 안에 위젯들을 묶고
with st.form("login"):
    email = st.text_input("이메일")
    pw = st.text_input(
        "비밀번호", type="password"
    )
    ok = st.form_submit_button("로그인")

if ok:
    verify(email, pw)
```

form 안 위젯은 제출 전까지 앱을 재실행하지 않습니다

→ 입력이 많은 화면(설문·로그인·필터 조합)에서
성능과 UX가 훨씬 좋아집니다.

데이터 표시 — DataFrame · Metric · JSON

pandas DataFrame을 그대로 넘기면 정렬·검색·페이지네이션이 알아서 됩니다.

data_display.py

```
import pandas as pd
df = pd.read_csv("power.csv")

# ① 인터랙티브 표 — 정렬/검색 가능 ★
st.dataframe(df, width="stretch")

# ② 정적 표 — 스타일 적용 가능
st.table(df.head(5))

# ③ KPI — 큰 숫자 + 증감
st.metric(
    label="이번 달 사용량",
    value="1,250 kWh",
    delta="+120 kWh"
)

# ④ JSON · dict 보기
st.json({"name": "홍길동", "team": "개발"})
```

PREVIEW · st.metric

이번 달 사용량

1,250 kWh

▲ +120 kWh

delta가 양수면 초록 ▲ · 음수면 빨강 ▼

PREVIEW · st.dataframe

date	region	usage
2026-01	광주	1,250
2026-02	광주	1,180
2026-03	전남	980
2026-04	전남	1,420

차트 — 한 줄로 그리는 3단계

단순 → 표준 → 인터랙티브. 데이터의 모양에 맞춰 선택하세요.

LEVEL 1 · 빌트인

빠르게 데이터 모양만 확인

```
st.line_chart(df)
st.bar_chart(df)
st.area_chart(df)
st.scatter_chart(df)
st.map(df) # lat, lon
```



st.bar_chart(df)

LEVEL 2 · Plotly ★

인터랙티브 — 줌·툴팁·범례

```
import plotly.express as px

fig = px.line(
    df, x="date", y="usage",
    color="region"
)
st.plotly_chart(fig)
```



st.plotly_chart(fig)

LEVEL 3 · 그 외

기존 코드 그대로 사용

```
# matplotlib · seaborn
st.pyplot(fig)

# altair
st.altair_chart(chart)

# 지도 — pydeck
st.pydeck_chart(deck)

# 그래프 — graphviz
st.graphviz_chart(dot)
```

추천 기본값

Plotly로 시작하세요. 인터랙티브 + 코드량 모두 균형이 좋습니다.

레이아웃 — columns · tabs · sidebar · expander

수직으로만 쌓이는 기본 레이아웃을 — 4가지 도구로 **2차원 화면**으로 만듭니다.

layout.py

```
# ① 좌우 분할 — columns
col1, col2 = st.columns(2)
with col1: st.metric("수익", "₩1.2M")
with col2: st.metric("고객", "248")

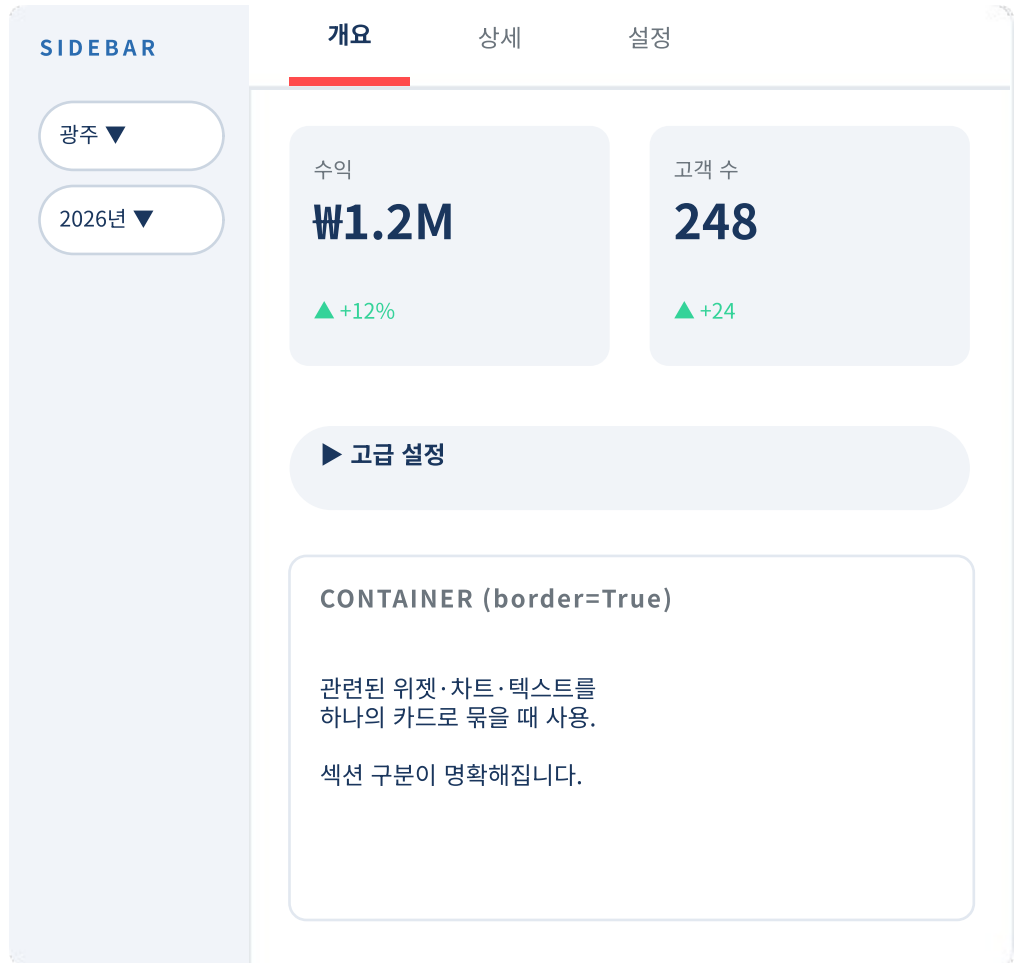
# 비율도 가능 — [3, 1]은 3:1
left, right = st.columns([3, 1])

# ② 탭 전환 — tabs
t1, t2, t3 = st.tabs(["개요", "상세", "설정"])

# ③ 좌측 사이드바 — 필터·메뉴에 단골
with st.sidebar:
    region = st.selectbox("지역", regions)

# ④ 접히는 상세 — expander
with st.expander("고급 설정"):
    st.checkbox("디버그 모드")

# ⑤ 컨테이너 그룹 (with 블록)
with st.container(border=True):
    st.write("카드 묶음")
```



상태 유지 · 파일 업로드 · 캐싱

"새로고침되면 값이 사라진다" 문제를 푸는 3가지 — Streamlit 중급의 핵심.

st.session_state — 값 기억하기 ★

```
# ① 초기화 (한 번만 실행되게)
if "count" not in st.session_state:
    st.session_state.count = 0

# ② 읽기 · 쓰기 (dict처럼 또는 .속성)
if st.button("+1"):
    st.session_state.count += 1
st.write(st.session_state.count)

# ③ 챗봇 대화 기록 — 가장 흔한 패턴
if "messages" not in st.session_state:
    st.session_state.messages = []

prompt = st.chat_input("질문하세요")
if prompt:
    st.session_state.messages.append(
        {"role": "user", "content": prompt}
    )
```

st.file_uploader — 파일 받기

```
file = st.file_uploader(
    "CSV 업로드", type=["csv", "xlsx"]
)

if file:
    df = pd.read_csv(file)
    st.dataframe(df)
```

@st.cache_data — 결과 재사용

위젯 조작마다 파일 다시 읽는 건 낭비.

같은 인자로 호출되면 **결과를 기억**해 즉시 반환.

```
@st.cache_data
def load_data(path):
    return pd.read_csv(path)

# DB 연결 · 모델 같은 공유 자원 → @st.cache_resource
```

DAY TWO · 8 HOURS

02

백엔드 · DB · 인증 풀스택 통합

→ 오늘 끝에는 — 1일차 React 앱이 DB와 로그인까지 연결된 진짜 서비스가 됩니다.

TODAY'S SCHEDULE

09:00 — 12:00

Part 1. REST API & Node.js 백엔드

3H · Express · CRUD

13:00 — 16:00

Part 2. DB 연동 & 프론트-백 통합

3H · Supabase · RLS

16:00 — 18:00

Part 3. 인증·인가 JWT / OAuth

2H · 로그인·회원가입

DAY 2 · PART 1

01

/ 03 parts

09:00 — 12:00

3시간 · API 설계

PART 1.

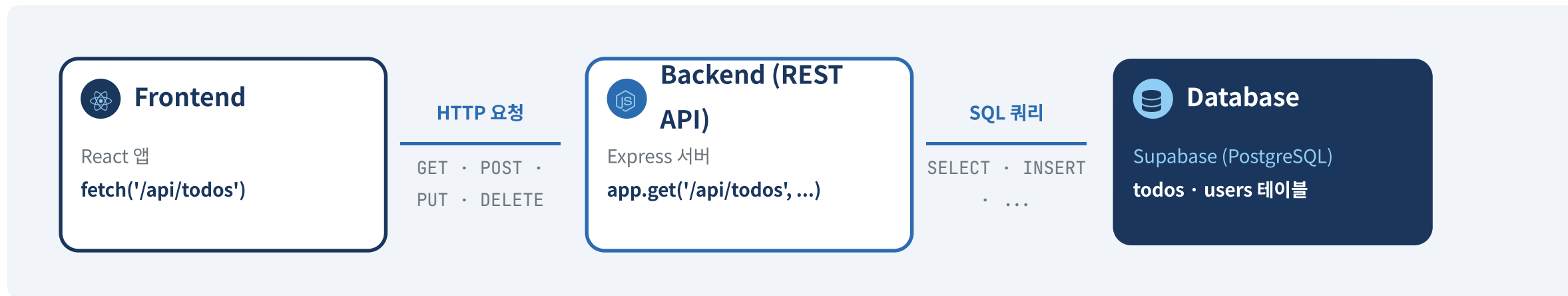
REST API 설계 & Node.js 백엔드 구현

LEARNING OUTCOMES

- ✓ REST 설계 원칙과 HTTP 메서드
- ✓ Express 서버로 CRUD API 구축
- ✓ 미들웨어 · 에러 처리 패턴
- ✓ Cursor AI로 백엔드 코드 자동 생성

REST API · 프론트와 백을 잇는 약속

웹에서 데이터를 주고받는 표준 방식 — HTTP 메서드로 리소스를 다룹니다.



REST의 4가지 원칙

01

리소스 = URL

데이터를 URL로 식별.

`/api/todos` · `/api/users/3`

02

동사 = HTTP 메서드

URL이 아니라 메서드로 동작 표현.

`GET` · `POST` · `PUT` · `DELETE`

03

JSON 형식

요청 · 응답 본문은 JSON.

`{"id":1, "title":"공부"}`

04

상태 비저장

요청마다 독립.

이전 요청을 서버가 기억 X → 토큰 전달.

HTTP 메서드 ↔ CRUD 매핑

4가지 메서드 · 4가지 CRUD 동작 · 6가지 핵심 상태 코드.

HTTP 메서드	CRUD	URL 예시	동작
GET	Read	/api/todos	전체 조회
POST	Create	/api/todos	새 할 일 생성
PUT	Update	/api/todos/:id	특정 할 일 수정
DELETE	Delete	/api/todos/:id	특정 할 일 삭제

HTTP STATUS CODES

기억할 6가지

200

OK – 조회 성공

201

Created – 생성 성공

400

Bad Req. – 잘못된 요청

401

Unauth. – 인증 필요

404

Not Found – 리소스 없음

500

Server Err – 서버 오류

AI에게 API 설계를 한 번에 받기

"할 일 관리 앱의 REST API를 설계해줘. 리소스: todos, users / 기능: CRUD + 인증 / 인증: JWT.
각 엔드포인트에 대해 HTTP 메서드 + URL + 요청 Body + 응답 예시 + 인증 필요 여부를 마크다운 표로 정리해줘."

→ AI가 즉시 8~12개 엔드포인트를 표로 정리해 줍니다. 그것을 그대로 Express 코드로 구현하면 끝.

Express 서버 — 30줄로 띄우기

npm 설치 한 번, 코드 30줄. 5분이면 API 서버가 동작합니다.

server.js · Express CRUD 골격

```
// ① 패키지 가져오기
import express from 'express';
import cors from 'cors';

// ② 미들웨어 등록
const app = express();
app.use(cors()); // 프론트 도메인 허용
app.use(express.json()); // JSON body 파싱

// ③ GET /api/todos - 전체 조회
app.get('/api/todos', (req, res) => {
  res.json({ todos: [] });
});

// ④ POST /api/todos - 새 할 일 추가
app.post('/api/todos', (req, res) => {
  const { title } = req.body;
  res.status(201).json({ id: Date.now(), title });
});

// ⑤ 서버 시작
app.listen(3001, () => console.log('→ :3001'));
```

5단계 해설

- ① 패키지 import**
npm i express cors · package.json에 "type": "module"
- ② 미들웨어 등록**
CORS + JSON 파서. 모든 요청에 자동 적용.
- ③ GET — 데이터 조회**
app.get(URL, handler) → res.json()으로 응답.
- ④ POST — 데이터 생성**
req.body로 입력 받고 201 Created 반환.
- ⑤ app.listen(포트)**
3001 포트에서 대기. node server.js로 실행.

테스트

Thunder Client(VS Code 확장) 또는 Postman으로 API 호출 → 응답 확인.

미들웨어 5종 + 실습

요청과 응답 사이에 끼어드는 함수. 한 번 등록하면 모든 라우트에 자동 적용됩니다.

01 · CROSS-ORIGIN

`cors()`

프론트 도메인 (예: localhost:5173)에서 API에 접근하도록 허용.

```
app.use(cors())
```

02 · BODY PARSER

`express.json()`

POST/PUT 요청 본문(JSON)을 자동으로 req.body로 파싱.

```
app.use(express.json())
```

03 · AUTH

`verifyToken`

JWT 토큰을 검증.
유효하지 않으면 401 반환 → 다음 핸들러로 진행 차단.

04 · ERROR HANDLER

`errorHandler`

전역 에러 처리.
throw된 모든 에러를 잡아 500 응답으로 변환.

05 · LOG

`morgan`

요청/응답 로그 자동 기록.
디버깅에 필수.

실습 · 1시간 핸즈온

AI에게 Express + Todo CRUD 백엔드 자동 생성 요청

① Agent 프롬프트

"Express + TypeScript로 Todo CRUD API 만들어줘. cors / json 미들웨어, 에러 핸들러 포함. 인메모리 배열로 임시 저장."

② 동작 검증 (Thunder Client)

POST /api/todos → 추가
GET /api/todos → 목록 확인
PUT /api/todos/1 → 수정
DELETE /api/todos/1 → 삭제

DAY 2 · PART 2

02

/ 03 parts

13:00 — 16:00

3시간 · DB 연동

PART 2.

DB 연동 & 프론트-백엔드 통합

LEARNING OUTCOMES

Supabase 프로젝트 생성 + 테이블 설계

RLS(Row Level Security)로 데이터 보안

Supabase JS 클라이언트로 CRUD

1일차 React 앱 ↔ Supabase 통합

Supabase — 백엔드 5분 컷

오픈소스 Firebase 대안 — DB · 인증 · 스토리지 · 실시간을 한 번에 제공하는 BaaS.

WHY SUPABASE FOR THIS COURSE

왜 Supabase인가

PostgreSQL 기반

진짜 SQL DB. NoSQL 한계 없음. JOIN · 트랜잭션 다 됩니다.

자동 REST API 생성

테이블을 만들면 → API 코드 0줄로 즉시 사용 가능.

인증 시스템 내장

이메일 · Google · Kakao OAuth — 추가 코드 없이 가능.

"1" 실시간 구독

데이터 변경 시 클라이언트에 자동 푸시 — WebSocket 직접 구현 X.

무료 티어

무료 플랜: 프로젝트 2개, 1주일 미사용 시 일시 중지(2026-10).

4-STEP SETUP

5분 안에 백엔드 구축

1

supabase.com 가입 + 프로젝트 생성

GitHub 계정으로 로그인. 지역은 Northeast Asia(Seoul) 권장.

2

URL + publishable key 복사

Settings → API Keys(또는 Connect) → .env에 저장.

3

npm install + 클라이언트 설치

```
npm install @supabase/supabase-js
```

4

supabase 클라이언트 초기화

```
import { createClient } from '@supabase/supabase-js';
export const supabase = createClient(URL, KEY);
→ 어디서든 import해서 바로 사용
```

테이블 설계 + RLS 정책

SQL Editor에서 3개 명령으로 — 테이블 + 보안 정책까지 한 번에 끝냅니다.

SQL Editor · todos 테이블 + RLS

-- ① 테이블 생성

```
CREATE TABLE todos (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID REFERENCES auth.users(id),
  title TEXT NOT NULL,
  completed BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT now()
);
```

-- ② RLS 활성화 (행 단위 보안)

```
ALTER TABLE todos ENABLE ROW LEVEL SECURITY;
```

-- ③ 본인 데이터만 접근 가능 정책

```
CREATE POLICY "Users manage own todos"
ON todos FOR ALL
USING (auth.uid() = user_id);
```

✓ 이제 이 테이블용 REST API를 바로 쓸 수 있습니다.

RLS · ROW LEVEL SECURITY

행 단위 보안이란?

DB 자체에서 권한을 검사합니다.

코드에서 if (user_id === ...) 검사할 필요가 없습니다.

정책을 만들어 두면 — 누가 어떤 방법으로 접근해도
본인 데이터만 볼 수 있습니다.

▲ 주의 — 매우 중요

RLS 없이는 모든 데이터가 공개됩니다.

Supabase는 클라이언트가 publishable key로 직접 접근합니다.
RLS를 끄면 누구나 다른 사람 데이터를 볼 수 있습니다.

→ 모든 테이블에 RLS는 기본값으로 켜두세요.

CRUD를 4줄로 — Supabase 클라이언트

SELECT / INSERT / UPDATE / DELETE — 각 한 줄로 끝납니다.

SELECT · 조회

```
// 전체 조회 + 최신순 정렬
const { data: todos } = await supabase
  .from('todos')
  .select('*')
  .order('created_at', { ascending: false });
```

INSERT · 생성

```
// 새 할 일 추가
await supabase
  .from('todos')
  .insert({
    title: '새 할 일',
    user_id: user.id
  });
```

UPDATE · 수정

```
// 완료 상태로 변경
await supabase
  .from('todos')
  .update({ completed: true })
  .eq('id', todoId);
// .eq(필드, 값) → WHERE 조건
```

DELETE · 삭제

```
// 특정 할 일 삭제
await supabase
  .from('todos')
  .delete()
  .eq('id', todoId);
// RLS가 자동으로 본인 데이터만 삭제 허용
```

DAY 2 · PART 3 · FINALE

03

/ 03 parts

16:00 — 18:00

2시간 · 로그인 완성

PART 3.

인증 · 인가 JWT & OAuth

TODAY'S DELIVERABLE

로그인이 되는 서비스

이메일 · 구글 · 카카오 로그인 + 보호된 라우트(AuthGuard) + 본인 데이터만 표시되는
진짜 서비스를 완성합니다.

🔒 인증/인가의 차이 — 누구인가 vs 무엇을 할 수 있는가

Authentication ≠ Authorization

자주 헷갈리지만 — 완전히 다른 두 가지입니다.

AUTHENTICATION · 인증

"너는 누구인가?"

로그인 · 회원가입.

이메일 · 비밀번호 · 소셜 계정으로 "이 사용자가 맞다"를 증명합니다.

예: Google 로그인 버튼 클릭 → 본인 인증

AUTHORIZATION · 인가

"무엇을 할 수 있나?"

권한 · 정책.

로그인한 사용자가 어떤 기능 · 데이터에 접근할 수 있는지 결정합니다.

예: 관리자만 회원 삭제 가능 · 본인 게시글만 수정

대표 인증 방식 3가지

METHOD 01 · 전통적

세션 기반

서버 메모리/DB에 세션 저장.

쿠키로 세션 ID 전달.

→ SPA · 모바일에선 비효율적.

METHOD 02 · 본 강의 ★

토큰 기반 · JWT

서버는 토큰만 발행.

클라이언트가 토큰을 보관 · 전송.

→ 무상태 · 확장성 ↑

METHOD 03 · 본 강의 ★

OAuth 2.0 · 소셜

Google · Kakao 등 외부 ID 사용.

비밀번호 관리 불필요.

→ 사용자 가입률 ↑

JWT — 토큰은 3조각으로 구성됩니다

Header.Payload.Signature — 점(.)으로 구분되는 문자열입니다.

JWT TOKEN · 실제 형태

eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoiaMTIzIn0.xWzZ_Q3Vk...signature

① HEADER · 헤더

알고리즘 + 토큰 타입

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

② PAYLOAD · 본문

사용자 정보 (Claims)

```
{  
  "user_id": "123",  
  "email": "u@example.com",  
  "exp": 1717200000  
}
```

③ SIGNATURE · 서명

위·변조 방지 인장

```
HMACSHA256(  
  base64(header) + "." +  
  base64(payload),  
  SECRET_KEY)
```

→ JWT 사용 흐름 (4단계)

① 로그인 → ② 서버가 JWT 발급 → ③ 클라이언트 저장 (localStorage) → ④ 매 요청 Authorization 헤더에 첨부

Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoiaMTIzIn0.xxx

Supabase Auth — 한 줄 로그인

JWT 발행·저장·검증 — Supabase가 다 해줍니다. 우리는 한 줄만 호출하면 됩니다.

✉ EMAIL · 이메일 + 비밀번호

```
// ① 회원가입
await supabase.auth.signUp({
  email: 'user@example.com',
  password: 'securePassword123'
});

// ② 로그인
await supabase.auth.signInWithPassword({
  email, password
});

// ③ 현재 사용자 확인
const { data: { user } } =
await supabase.auth.getUser();

// ④ 로그아웃
await supabase.auth.signOut();

// ⑤ 인증 상태 변경 구독
supabase.auth.onAuthStateChange(
  (event, session) => console.log(event));
```

🔗 OAUTH · Google / Kakao 소셜 로그인

Google 로그인

```
await supabase.auth.signInWithOAuth({
  provider: 'google',
  options: {
    redirectTo: window.location.origin
  }
});
```

Kakao 로그인 — provider만 교체

```
await supabase.auth.signInWithOAuth({
  provider: 'kakao'
});
```

⚙️ 사전 설정

Supabase Dashboard → Authentication → Providers → Google/Kakao 활성화
+ Client ID 입력.

AuthContext + 보호된 라우트

앱 전역에서 로그인 상태를 공유하고, 비로그인 사용자는 로그인 페이지로 보냅니다.

AuthContext.tsx · 전역 인증 상태

```
export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(undefined);
  useEffect(() => {
    // 시작 시 현재 세션 확인
    supabase.auth.getSession().then(({ data }) => setUser(data.session?.user ?? null));
    // 인증 변경 구독
    const { data: sub } = supabase.auth.onAuthStateChange(_, s) => setUser(s?.user ?? null);
    return () => sub.subscription.unsubscribe();
  }, []);
  return <AuthContext.Provider value={{ user }}>{children}</AuthContext.Provider>;
};
```

AuthGuard · 보호된 라우트

```
const AuthGuard = ({ children }) => {
  const { user } = useAuth();
  if (user === undefined) return <Loading/>;
  if (!user)
    return <Navigate to="/login" />;
  return children;
};
// <Route
element={<AuthGuard><MyPage /></AuthGuard>} />
```

실습 · 2일차 통합 결과물

1일차 Todo 앱에 로그인 + 본인 데이터 분리 적용

1 로그인 페이지

Email + Google + Kakao 버튼 3개 · 회원가입 폼 · 에러 메시지

2 AuthGuard 적용

/todos · /mypage 라우트를 AuthGuard로 감싸 비로그인 차단

3 RLS 동작 확인

두 계정으로 로그인해 보고
— 각자 본인 할 일만 보이는지 검증

DAY THREE · 5 HOURS · FINALE

03

Streamlit · 데이터 · AI 챗봇 배포

→ 마지막 — 작품 2개(웹 + Streamlit)를 친구·동료에게 URL로 공유합니다.

TODAY'S SCHEDULE

09:00 — 11:00

Part 1. Streamlit & Python 기초

2H · 자기소개 앱

11:00 — 12:00

Part 2. 데이터 앱 개발

1H · 전력 데이터 대시보드

13:00 — 15:00

Part 3. AI 연동 & Cloud 배포

2H · Claude API · 발표

DAY 3 · PART 1

01

/ 03 parts

09:00 — 11:00

2시간 · Python 입문

PART 1.

Streamlit 소개 & Python 기초

CONCEPT SHIFT

"HTML/CSS/JS 없이도
웹앱을 만들 수 있습니다."

Python 코드만으로 인터랙티브 웹앱을 구축 — 데이터 분석가·기획자도 활용 가능한 도구.

⚡ 저장하면 즉시 브라우저 반영 — 핫 리로드 지원

Streamlit — 파이썬으로 만드는 웹앱

데이터 분석가·기획자·비개발자도 — Python 코드만으로 인터랙티브 웹앱을 구축할 수 있습니다.

기존 웹 개발

React + Express + DB

HTML · CSS · JS · 백엔드 · 인증 · 배포 — 전부 따로 학습해야 합니다.

🕒 러닝 커브 가파름 — 풀스택 한 주에 안 끝남

STREAMLIT 방식

Python 파일 1개

app.py 하나에 Python으로 다 작성. 웹 지식 0이어도 시작 가능.

⚡ 3시간이면 첫 데이터 앱 배포

Streamlit 5대 강점



Python only

HTML/CSS/JS 지식 없이
Python만으로 웹앱 생성.



핫 리로드

코드 저장 → 브라우저 즉시 반영.
빌드·새로고침 없음.



풍부한 위젯

slider · selectbox ·
file_uploader 등
즉시 사용.



데이터 호환

pandas · matplotlib · plotly 와
완벽 호환.



무료 배포

Streamlit Cloud로
GitHub 연동
→ 무료 호스팅.

Python — 30분 핵심 문법 6개

JavaScript와 거의 같은 개념 — 문법만 살짝 다릅니다. 6개면 충분합니다.

01 · 변수

선언 키워드 X

```
name = "홍길동" # 문자열
count = 100 # 숫자
```

02 · 리스트

[] — JS 배열과 동일

```
items = ["전력", "IT", "보안"]
items.append("AI")
```

03 · 딕셔너리

{ } — JS 객체

```
data = { "부서": "전력IT",
         "인원": 50 }
```

04 · 함수

def 키워드

```
def calc(x, y):
    return x + y
```

05 · 조건문

if · 들여쓰기 = { }

```
if value > 100:
    print("초과")
```

06 · 반복문

for · in

```
for item in items:
    print(item)
```

↔ JS와 다른 점 — 4가지만

- ① 세미콜론(;) 없음 · 줄바꿈으로 구분 ② 중괄호({}) 대신 들여쓰기 (4칸)
- ③ 함수는 def, 화살표 없음 · 단순함 ④ true/false 대신 True/False (대문자)

첫 Streamlit 앱 + 자기소개 실습

설치 한 줄 · 코드 10줄 · streamlit run 한 번. 끝.

설치 + 첫 실행

```
# 1. 설치 (한 줄)
$ pip install streamlit

# 2. app.py 파일 만들고 실행
$ streamlit run app.py

→ 브라우저 자동 오픈: http://localhost:8501
# 코드 저장하면 즉시 새로고침
```

기본 컴포넌트 5개

```
st.title("앱 제목") # 큰 제목
st.write("내용") # 텍스트/데이터
st.button("클릭") # 버튼
st.text_input("이름") # 입력칸
st.selectbox("부서", opts) # 드롭다운
```

실습 · 자기소개 Streamlit 앱

이름·부서·관심분야를 입력받아 화면에 표시하기

```
# app.py
import streamlit as st
st.title("안녕하세요")
name = st.text_input("이름")
dept = st.selectbox("부서", ["전력IT", "보안", "AI"])
st.write(f"안녕하세요 {name}님 ({dept})")
```

AI 프롬프트로 한 번에 받기

"Streamlit으로 자기소개 앱 만들어줘. 이름·부서·관심분야 입력 폼, 입력하면 카드 형태로 표시. 프로필 이미지 업로드 기능 + 사이드바에 사진 미리보기 추가."

DAY 3 · PART 2

02

/ 03 parts

11:00 — 12:00

1시간 · 데이터 앱

PART 2.

Streamlit 데이터 앱 개발

TODAY'S DELIVERABLE

전력 데이터 대시보드

CSV를 업로드하면

— 월별 추이 · 지역별 비교 · 핵심 KPI가 자동으로 시각화되는 인터랙티브 대시보드를 한 시간 안에 만듭니다.

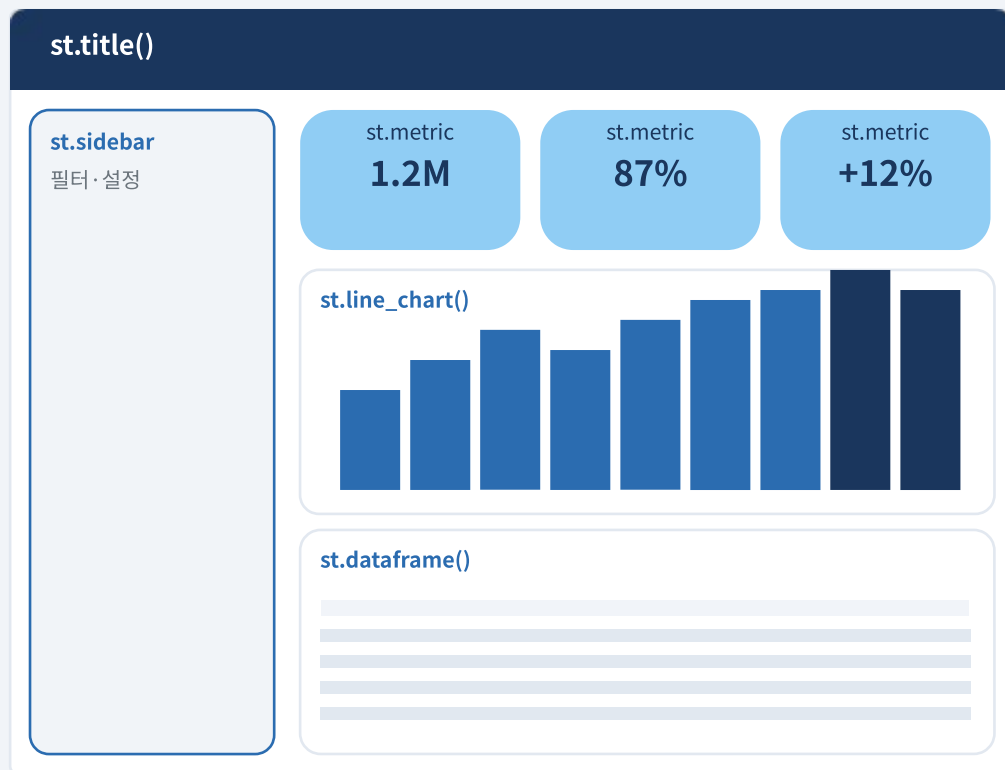
pandas · matplotlib · plotly — 3대 도구 통합

레이아웃 + 위젯 8종

사이드바·컬럼·탭으로 화면을 나누고, 8개 위젯으로 사용자 입력을 받습니다.

레이아웃 구조 미리보기

대시보드 표준 구조



8 ESSENTIAL WIDGETS

필수 위젯 8개

- 01 `st.sidebar` – 사이드바
- 02 `st.columns(N)` – N등분 분할
- 03 `st.slider` – 숫자 범위
- 04 `st.selectbox` – 드롭다운 선택
- 05 `st.multiselect` – 복수 선택
- 06 `st.file_uploader` – CSV/Excel 업로드
- 07 `st.metric` – KPI 카드
- 08 `st.tabs` · `st.expander` – 콘텐츠 정리

CSV 업로드 → 분석 → 시각화

pandas로 데이터를 다루고, Streamlit 차트 한 줄로 그립니다.

pandas · 데이터 다루기

```
# CSV 읽기
import pandas as pd
df = pd.read_csv("power.csv")

# 기본 통계량
df.describe() # 평균·중앙값·표준편차

# 그룹별 합산
df.groupby("지역").sum()
```

Streamlit 차트 (한 줄!)

```
st.line_chart(df) # 라인 차트
st.bar_chart(df) # 막대 차트
st.area_chart(df) # 영역 차트
st.map(df) # 지도 (lat/lon)
st.dataframe(df) # 데이터 테이블
st.plotly_chart(fig) # 고급 차트
```

실습 · 전력 데이터 대시보드 (1H)

CSV 업로드 → 월별/지역별 자동 분석 대시보드

기능 요구사항

- ✓ st.file_uploader로 전력 CSV 업로드
- ✓ 상단 KPI 4개 (총 사용량 · 평균 · 최대 · 증감)
- ✓ 월별 추이 라인 차트 + 지역별 막대 차트
- ✓ 사이드바 기간/지역 필터

AI 프롬프트 예시: "전력 사용량 CSV 대시보드를 만들어줘."

file_uploader 받고 → 상단 metric 4개 (총합·평균·최대·전월대비 증감%) → 월별 line_chart

→ 지역별 bar_chart → 사이드바에 기간/지역 multiselect 필터. 컬럼명: 날짜, 지역, 사용량."

DAY 3 · PART 3 · FINALE

03

/ 03 parts

13:00 — 15:00

2시간 · 최종 발표

PART 3.

AI 연동 & Cloud 배포

21시간의 마무리

URL로 공유 가능한 AI 데이터 앱

Claude API를 연동한 AI 챗봇 + Streamlit Cloud 무료 배포 + 최종 프로젝트 발표.

21시간이 끝나는 시점 — 친구·동료에게 자랑할 작품 2개

Claude API 연동 — Streamlit + AI

5단계로 — 데이터 앱에 "이 데이터에서 이상치 찾아줘" 같은 자연어 질의를 추가합니다.



ai_chatbot.py · Streamlit + Claude

```

import streamlit as st; from anthropic import Anthropic
client = Anthropic(api_key=st.secrets["ANTHROPIC_API_KEY"])
# 세션에 대화 이력 보관
if "messages" not in st.session_state: st.session_state.messages = []
# 사용자 입력 받기
user_input = st.chat_input("전력 데이터에 대해 물어보세요")
if user_input:
    st.session_state.messages.append({"role": "user", "content": user_input})
    resp = client.messages.create(model="claude-sonnet-5-5", messages=st.session_state.messages, max_tokens=1024)
    st.session_state.messages.append({"role": "assistant", "content": resp.content[0].text})
for m in st.session_state.messages: st.chat_message(m["role"]).write(m["content"])
  
```

Streamlit Cloud 무료 배포 — 5분

GitHub에 푸시하고 — share.streamlit.io에서 클릭 → 끝. 공유 가능한 URL이 발급됩니다.

1

코드 푸시

app.py를 GitHub 리포에 푸시. (public 또는 private 모두 가능)

2

requirements.txt

```
streamlit
pandas
plotly
anthropic
```

3

Cloud 연결

share.streamlit.io 접속 →
GitHub 로그인 →
"Create app" 클릭.

4

Secrets 등록

Advanced settings →
Secrets에 키 입력.
st.secrets[]로
안전하게 접근.

5

Deploy!

자동 빌드 → URL 발급.
이후 git push 시마다
자동 재배포.

무료 플랜 한도

Community Cloud 무료(2026-10)

- ✓ 무제한 public 앱 배포
- ✓ 메모리 최대 2.7GB · 12시간 무접속 시 슬립
- ✓ 신용카드 불필요
- ✓ 주소는 이름.streamlit.app(커스텀 도메인 없음)

최소 requirements.txt 예시

필수 패키지만 — 빠른 빌드

```
streamlit==1.65.0
```

```
pandas==3.0.6
```

```
plotly==7.1.0
```

```
anthropic==1.12.1
```

버전을 고정해두면 — 호환성 깨질 일 없음

최종 프로젝트 발표 (1시간)

21시간을 완주한 각자 — 자신의 작품 2개를 5분씩 시연합니다.

발표 구성 · 1인 5분

기획 → 기술 → 시연 → 회고

① 기획 의도 (1분)

어떤 문제를 풀려고 했나?
누구를 위해 만들었나?
우리 업무와 어떻게 연결되나?

② 기술 구성 (1분)

React/Streamlit · DB · 인증 · AI API
— 어떤 스택을 골랐고 왜 그렇게 했나?

③ 실제 시연 (2분) ★

배포된 URL 열기 → 핵심 기능 3가지
라이브 데모 → 청중의 입력으로
동작 확인

④ 회고 + Q&A (1분)

AI에게 가장 잘 시킨 일 · 가장
막혔던 부분 · 향후 확장 계획.

🏆 21시간 결과물 체크리스트

발표 전 — 이 5가지가 다 있는지 확인하세요

- ✓ ① 풀스택 웹앱 (React + Supabase) — 배포 완료
- ✓ ② Streamlit 데이터 앱 — Cloud에 배포 완료
- ✓ ③ 로그인 동작 (이메일 또는 OAuth)

- ✓ ④ AI API 연동 (Claude or ChatGPT) — 응답 확인
- ✓ ⑤ GitHub 리포 — README + 스크린샷 정리
- ★ 발표 = 5분 · QR로 URL 공유

PROJECT CATALOG · 24 IDEAS

바이브코딩으로 만들 수 있는 프로젝트 24선

4개 분야 × 6개 = 강의 후 본인이 도전해볼 수 있는 다음 프로젝트 리스트



CATEGORY 01

웹 개발

React + Supabase 풀스택.

6 선



CATEGORY 02

Streamlit

Python 데이터 앱.

6 선



CATEGORY 03

데이터 분석

실무 데이터 분석 도구.

6 선



CATEGORY 04

AI 앱

Claude API 활용.

6 선

CATEGORY 01 / 04



웹 개발

React · TypeScript · Supabase로 만드는 실서비스

6

PROJECTS

PROJECTS IN THIS CATEGORY

초급 · 30분
Todo 앱

React + TypeScript로 할 일 관리 앱. 입력 · 체크 · 삭제 · 필터링 + localStorage 데이터 유지.

React · TypeScript · CSS

중급 · 45분
포트폴리오 사이트

개인 포트폴리오 SPA. Hero · About · Skills · Projects · Contact 섹션 + 스크롤 애니메이션.

React · TypeScript · CSS

고급 · 60분
실시간 게시판

Supabase 백엔드 게시판. 회원가입 · 로그인 · 게시글 CRUD · 댓글 · 실시간 알림까지.

React · TypeScript · Supabase

중급 · 40분
랜딩 페이지 빌더

마케팅용 원페이지. Hero 배너 · 특징 소개 · 가격표 · CTA · FAQ 아코디언.

React · TypeScript · CSS

초급 · 35분
날씨 앱

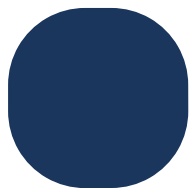
OpenWeatherMap API 연동. 도시별 현재 날씨 · 5일 예보 · 날씨 아이콘 · 반응형.

React · TypeScript · API

고급 · 60분
관리자 대시보드

Chart.js로 매출 · 사용자 · 주문 통계 시각화. 사이드바 네비 + 다크모드 토글.

React · TypeScript · Chart.js



PROJECT 01 · 웹 개발

Todo 앱

React + TypeScript로 할 일 관리 앱. 입력·체크·삭제·필터링 + localStorage 데이터 유지.

WHAT YOU'LL BUILD · 만들 것

1 입력창·체크박스·삭제 버튼 UI

2 useState 기반 필터링

3 localStorage 영속 저장

TECH STACK

React

TypeScript

CSS

핵심 팁

AI에게 '카테고리별 필터링이 가능한 Todo 앱'으로 요청하면 30분 안에 동작합니다.

STARTER PROMPT · Cursor에 그대로 붙여넣기

"Todo 앱 만들어줘. 기능: 입력창·체크박스·삭제 버튼 UI · useState 기반 필터링 · localStorage 영속 저장. 기술 스택: React + TypeScript + CSS. 깔끔한 UI · 반응형 · 주석 포함."



PROJECT 02 · 웹 개발

포트폴리오 사이트

개인 포트폴리오 SPA. Hero · About · Skills · Projects · Contact 섹션 + 스크롤 애니메이션.

🔧 WHAT YOU'LL BUILD · 만들 것

1 Hero · About · Skills · Projects · Contact 5섹션

2 Scroll Reveal 애니메이션

3 반응형 모바일 대응

📦 TECH STACK

React

TypeScript

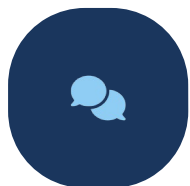
CSS

💡 핵심 팁

Figma 디자인을 첨부하면 Cursor가 화면별 컴포넌트로 자동 분리해 줍니다.

🔗 STARTER PROMPT · Cursor에 그대로 붙여넣기

"포트폴리오 사이트 만들어줘. 기능: Hero · About · Skills · Projects · Contact 5섹션 · Scroll Reveal 애니메이션 · 반응형 모바일 대응. 기술 스택: React + TypeScript + CSS. 깔끔한 UI · 반응형 · 주석 포함."



PROJECT 03 · 웹 개발

실시간 게시판

Supabase 백엔드 게시판. 회원가입·로그인·게시글 CRUD·댓글·실시간 알림까지.

🔧 WHAT YOU'LL BUILD · 만들 것

1 Supabase 회원가입·로그인

2 게시글 CRUD + 댓글 1:N

3 실시간 알림 (subscribe)

📦 TECH STACK

React

TypeScript

Supabase

💡 핵심 팁

1·2일차에서 배운 인증 + RLS 정책이 그대로 적용되는 종합 실습 프로젝트.

🔗 STARTER PROMPT · Cursor에 그대로 붙여넣기

"실시간 게시판 만들어줘. 기능: Supabase 회원가입·로그인 · 게시글 CRUD + 댓글 1:N · 실시간 알림 (subscribe). 기술 스택: React + TypeScript + Supabase. 깔끔한 UI · 반응형 · 주석 포함."



PROJECT 04 · 웹 개발

랜딩 페이지 빌더

마케팅용 원페이지. Hero 배너 · 특징 소개 · 가격표 · CTA · FAQ 아코디언.

🔧 WHAT YOU'LL BUILD · 만들 것

1 Hero 배너 + CTA 버튼

2 특징 3섹션 + 가격표

3 FAQ 아코디언 인터랙션

📦 TECH STACK

React

TypeScript

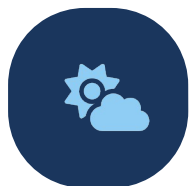
CSS

💡 핵심 팁

AI에게 '제품명 + 타겟 + 핵심 가치 3개'만 던지면 카피라이팅까지 자동 생성.

✍️ STARTER PROMPT · Cursor에 그대로 붙여넣기

"랜딩 페이지 빌더 만들어줘. 기능: Hero 배너 + CTA 버튼 · 특징 3섹션 + 가격표 · FAQ 아코디언 인터랙션. 기술 스택: React + TypeScript + CSS. 깔끔한 UI · 반응형 · 주석 포함."



PROJECT 05 · 웹 개발

날씨 앱

OpenWeatherMap API 연동 · 도시별 현재 날씨 · 5일 예보 · 날씨 아이콘 · 반응형.

🔧 WHAT YOU'LL BUILD · 만들 것

1 OpenWeatherMap API 호출

2 현재 날씨 + 5일 예보 카드

3 도시 검색 + 즐겨찾기

📦 TECH STACK

React

TypeScript

API

💡 핵심 팁

외부 API 호출 + async/await + 에러 처리 패턴을 자연스럽게 익힙니다.

🔗 STARTER PROMPT · Cursor에 그대로 붙여넣기

"날씨 앱 만들어줘. 기능: OpenWeatherMap API 호출 · 현재 날씨 + 5일 예보 카드 · 도시 검색 + 즐겨찾기. 기술 스택: React + TypeScript + API. 깔끔한 UI · 반응형 · 주석 포함."



PROJECT 06 · 웹 개발

관리자 대시보드

Chart.js로 매출·사용자·주문 통계 시각화. 사이드바 네비 + 다크모드 토글.

🔧 WHAT YOU'LL BUILD · 만들 것

1 Chart.js 매출·사용자·주문 차트

2 사이드바 네비게이션

3 다크/라이트 테마 토글

📦 TECH STACK

React

TypeScript

Chart.js

💡 핵심 팁

데이터 시각화 + 테마 시스템 + 권한별 메뉴까지 — 어드민 화면의 정석.

✍️ STARTER PROMPT · Cursor에 그대로 붙여넣기

"관리자 대시보드 만들어줘. 기능: Chart.js 매출·사용자·주문 차트 · 사이드바 네비게이션 · 다크/라이트 테마 토글. 기술 스택: React + TypeScript + Chart.js. 깔끔한 UI · 반응형 · 주석 포함."

CATEGORY 02 / 04



Streamlit 앱

Python만으로 빠르게 만드는 데이터 앱

6

PROJECTS

PROJECTS IN THIS CATEGORY



중급 · 30분
전력 데이터 대시보드

CSV 업로드 → 전력사용량 데이터를 월별·지역별로 시각화, 사이드바 필터.

Streamlit · pandas · plotly



초급 · 25분
CSV 데이터 탐색기

어떤 CSV든 업로드하면 자동으로 데이터 요약·통계·시각화를 제공하는 범용 도구.

Streamlit · pandas · plotly



초급 · 15분
자기소개 앱

Streamlit 첫 앱. 이름·사진·경력·스킬 바·취미를 표시하는 인터랙티브 페이지.

Streamlit



중급 · 30분
업무 일정 관리 앱

일일 업무 일정 작성·저장. 날짜별 조회·카테고리 분류·JSON 저장.

Streamlit · pandas · json



초급 · 20분
이미지 갤러리 뷰어

이미지 업로드 → 갤러리 표시 + 필터(밝기/대비/흑백) 적용.

Streamlit · Pillow



중급 · 35분
가계부 앱

수입·지출 기록 + 월별 통계 + 카테고리별 지출 차트.

Streamlit · pandas · plotly



PROJECT 01 · Streamlit 앱

전력 데이터 대시보드

CSV 업로드 → 전력사용량 데이터를 월별·지역별로 시각화. 사이드바 필터.

🔧 WHAT YOU'LL BUILD · 만들 것

- 1 CSV 업로드 위젯
- 2 월별 추이 + 지역별 비교 차트
- 3 사이드바 필터링

📦 TECH STACK

Streamlit

pandas

plotly

💡 핵심 팁

Day 3 Part 2 실습 그대로. 본인 부서의 실제 데이터로 시도해보세요.

🔗 STARTER PROMPT · Cursor에 그대로 붙여넣기

"전력 데이터 대시보드 만들어줘. 기능: CSV 업로드 위젯 · 월별 추이 + 지역별 비교 차트 · 사이드바 필터링. 기술 스택: Streamlit + pandas + plotly. 깔끔한 UI · 반응형 · 주석 포함."



PROJECT 02 · Streamlit 앱

CSV 데이터 탐색기

어떤 CSV든 업로드하면 자동으로 데이터 요약·통계·시각화를 제공하는 범용 도구.

🔧 WHAT YOU'LL BUILD · 만들 것

- 1 어떤 CSV든 자동 감지
- 2 describe() + missing values 표시
- 3 컬럼별 자동 시각화

📦 TECH STACK

Streamlit

pandas

plotly

💡 핵심 팁

한 번 만들어 두면 — 새 데이터를 받을 때마다 분석이 자동으로.

✍️ STARTER PROMPT · Cursor에 그대로 붙여넣기

"CSV 데이터 탐색기 만들어줘. 기능: 어떤 CSV든 자동 감지 · describe() + missing values 표시 · 컬럼별 자동 시각화. 기술 스택: Streamlit + pandas + plotly. 깔끔한 UI · 반응형 · 주석 포함."



PROJECT 03 · Streamlit 앱

자기소개 앱

Streamlit 첫 앱. 이름·사진·경력·스킬 바·취미를 표시하는 인터랙티브 페이지.

🔧 WHAT YOU'LL BUILD · 만들 것

1 이름·부서·관심분야 입력

2 프로필 사진 업로드

3 스킬 진행률 바

📦 TECH STACK

Streamlit

💡 핵심 팁

Streamlit 위젯 5개만 익히면 만들 수 있는 — 가장 빠른 첫 성공 경험.

🔗 STARTER PROMPT · Cursor에 그대로 붙여넣기

"자기소개 앱 만들어줘. 기능: 이름·부서·관심분야 입력 · 프로필 사진 업로드 · 스킬 진행률 바. 기술 스택: Streamlit. 깔끔한 UI · 반응형 · 주석 포함."



PROJECT 04 · Streamlit 앱

업무 일정 관리 앱

일일 업무 일정 작성·저장. 날짜별 조회 · 카테고리 분류 · JSON 저장.

🔧 WHAT YOU'LL BUILD · 만들 것

- 1 일자별 업무 입력 + 저장
- 2 카테고리 분류 (회의·개발·문서)
- 3 JSON 영속 + 캘린더 뷰

📦 TECH STACK

Streamlit

pandas

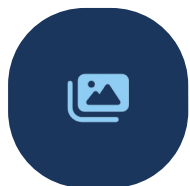
json

💡 핵심 팁

개인 업무 관리 + 사내 보고서 작성 보조 도구로 활용 가능.

✍️ STARTER PROMPT · Cursor에 그대로 붙여넣기

"업무 일정 관리 앱 만들어줘. 기능: 일자별 업무 입력 + 저장 · 카테고리 분류 (회의·개발·문서) · JSON 영속 + 캘린더 뷰. 기술 스택: Streamlit + pandas + json. 깔끔한 UI · 반응형 · 주석 포함."



PROJECT 05 · Streamlit 앱

이미지 갤러리 뷰어

이미지 업로드 → 갤러리 표시 + 필터(밝기/대비/흑백) 적용.

🔧 WHAT YOU'LL BUILD · 만들 것

- 1 이미지 다중 업로드
- 2 갤러리 그리드 표시
- 3 밝기/대비/흑백 슬라이더

📦 TECH STACK

Streamlit

Pillow

💡 핵심 팁

이미지 처리 기초 + Streamlit 인터랙티브 컨트롤을 결합한 미니 프로젝트.

🔗 STARTER PROMPT · Cursor에 그대로 붙여넣기

"이미지 갤러리 뷰어 만들어줘. 기능: 이미지 다중 업로드 · 갤러리 그리드 표시 · 밝기/대비/흑백 슬라이더. 기술 스택: Streamlit + Pillow. 깔끔한 UI · 반응형 · 주석 포함."



PROJECT 06 · Streamlit 앱

가계부 앱

수입·지출 기록 + 월별 통계 + 카테고리별 지출 차트.

🔧 WHAT YOU'LL BUILD · 만들 것

- 1 수입/지출 입력 폼
- 2 월별 잔액 + 카테고리별 차트
- 3 거래 내역 테이블

📦 TECH STACK

Streamlit

pandas

plotly

💡 핵심 팁

데이터 추가 → 자동 집계 → 시각화 — 가장 자주 쓰이는 패턴 학습.

🔗 STARTER PROMPT · Cursor에 그대로 붙여넣기

"가계부 앱 만들어줘. 기능: 수입/지출 입력 폼 · 월별 잔액 + 카테고리별 차트 · 거래 내역 테이블. 기술 스택: Streamlit + pandas + plotly. 깔끔한 UI · 반응형 · 주식 포함."



데이터 분석

실무 데이터를 자동으로 분석하는 도구

6

PROJECTS

PROJECTS IN THIS CATEGORY



중급 · 35분
설비 장애 분석기

전력 설비 장애 이력 Excel 데이터를 분석. 유형별 빈도 · 월별 추이 · 지역별 분포 · 복구시간 통계.

Streamlit · pandas · plotly



중급 · 30분
Excel 자동 보고서

Python으로 Excel 데이터를 읽어 요약 통계 · 차트 · 서식 적용된 보고서를 자동 생성.

Streamlit · pandas · openpyxl



중급 · 30분
설문 결과 분석기

설문조사 CSV 업로드 → 응답 분포 · 교차 분석 · 워드 클라우드 자동 생성.

Streamlit · pandas · plotly



초급 · 25분
매출 데이터 분석

회사 매출 CSV 업로드 → 월별 추이 · 제품별 점유율 · 전년 대비 성장률 · 예측 트렌드.

Streamlit · pandas · plotly



중급 · 35분
인사 데이터 대시보드

직원 인사 데이터 분석. 부서별 인원 · 연차 분포 · 성별 비율 · 퇴직률 추이.

Streamlit · pandas · plotly



고급 · 40분
웹 로그 분석기

웹 서버 접속 로그 분석. 시간대별 트래픽 · 인기 페이지 · 브라우저 분포 · 에러율 시각화.

Streamlit · pandas · plotly



PROJECT 01 · 데이터 분석

설비 장애 분석기

전력 설비 장애 이력 Excel 데이터를 분석. 유형별 빈도 · 월별 추이 · 지역별 분포 · 복구시간 통계.

🔧 WHAT YOU'LL BUILD · 만들 것

1 Excel 장애 이력 업로드

2 유형·지역별 빈도 분석

3 복구시간 box plot

📦 TECH STACK

Streamlit

pandas

plotly

💡 핵심 팁

전력·설비 실무와 가까운 분석 시나리오 — 실제 운영 데이터로 즉시 활용.

🔗 STARTER PROMPT · Cursor에 그대로 붙여넣기

"설비 장애 분석기 만들어줘. 기능: Excel 장애 이력 업로드 · 유형·지역별 빈도 분석 · 복구시간 box plot. 기술 스택: Streamlit + pandas + plotly. 깔끔한 UI · 반응형 · 주석 포함."



PROJECT 02 · 데이터 분석

Excel 자동 보고서

Python으로 Excel 데이터를 읽어 요약 통계·차트·서식 적용된 보고서를 자동 생성.

🔧 WHAT YOU'LL BUILD · 만들 것

- 1 Excel 시트 읽기
- 2 요약 통계 자동 계산
- 3 openpyxl로 서식 포함 출력

📦 TECH STACK

Streamlit

pandas

openpyxl

💡 핵심 팁

매주 반복하는 보고서 작업을 단 한 번의 클릭으로 — 가장 직접적인 시간 절약.

🔗 STARTER PROMPT · Cursor에 그대로 붙여넣기

"Excel 자동 보고서 만들어줘. 기능: Excel 시트 읽기 · 요약 통계 자동 계산 · openpyxl로 서식 포함 출력. 기술 스택: Streamlit + pandas + openpyxl. 깔끔한 UI · 반응형 · 주식 포함."



PROJECT 03 · 데이터 분석

설문 결과 분석기

설문조사 CSV 업로드 → 응답 분포 · 교차 분석 · 워드 클라우드 자동 생성.

🔧 WHAT YOU'LL BUILD · 만들 것

- 1 설문 CSV 업로드
- 2 응답 분포 + 교차 분석
- 3 워드클라우드 자동 생성

📦 TECH STACK

Streamlit

pandas

plotly

💡 핵심 팁

HR/만족도 설문 후 즉시 분석
— 회의 자료를 자동으로 만들어주는 도구.

✍️ STARTER PROMPT · Cursor에 그대로 붙여넣기

"설문 결과 분석기 만들어줘. 기능: 설문 CSV 업로드 · 응답 분포 + 교차 분석 · 워드클라우드 자동 생성. 기술 스택: Streamlit + pandas + plotly. 깔끔한 UI · 반응형 · 주석 포함."



PROJECT 04 · 데이터 분석

매출 데이터 분석

회사 매출 CSV 업로드 → 월별 추이 · 제품별 점유율 · 전년 대비 성장률 · 예측 트렌드.

🔧 WHAT YOU'LL BUILD · 만들 것

1 매출 데이터 시계열 분석

2 제품·지역별 점유율

3 전년 동기 대비 성장률

📦 TECH STACK

Streamlit

pandas

plotly

💡 핵심 팁

Pandas의 groupby · resample 기능을 자연스럽게 익히는 베스트 입문 프로젝트.

🔗 STARTER PROMPT · Cursor에 그대로 붙여넣기

"매출 데이터 분석 만들어줘. 기능: 매출 데이터 시계열 분석 · 제품·지역별 점유율 · 전년 동기 대비 성장률. 기술 스택: Streamlit + pandas + plotly. 깔끔한 UI · 반응형 · 주석 포함."



PROJECT 05 · 데이터 분석

인사 데이터 대시보드

직원 인사 데이터 분석. 부서별 인원 · 연차 분포 · 성별 비율 · 퇴직률 추이.

🔧 WHAT YOU'LL BUILD · 만들 것

1 부서별 인원 + 연차 분포

2 성별 비율 도넛

3 퇴직률 시계열 추이

📦 TECH STACK

Streamlit

pandas

plotly

💡 핵심 팁

데이터 식명화 + 다양한 차트 결합
— 임원 보고용 대시보드의 정석.

✍️ STARTER PROMPT · Cursor에 그대로 붙여넣기

"인사 데이터 대시보드 만들어줘. 기능: 부서별 인원 + 연차 분포 · 성별 비율 도넛 · 퇴직률 시계열 추이. 기술 스택: Streamlit + pandas + plotly. 깔끔한 UI · 반응형 · 주석 포함."



PROJECT 06 · 데이터 분석

웹 로그 분석기

웹 서버 접속 로그 분석. 시간대별 트래픽 · 인기 페이지 · 브라우저 분포 · 에러율 시각화.

🔧 WHAT YOU'LL BUILD · 만들 것

- 1 로그 정규식 파싱
- 2 시간대별 트래픽 + TopN 페이지
- 3 에러율 · 브라우저 분포

📦 TECH STACK

Streamlit

pandas

plotly

💡 핵심 팁

정규표현식으로 로그 파싱 + 시계열 집계
— 운영 모니터링의 핵심 패턴.

✍️ STARTER PROMPT · Cursor에 그대로 붙여넣기

"웹 로그 분석기 만들어줘. 기능: 로그 정규식 파싱 · 시간대별 트래픽 + TopN 페이지 · 에러율 · 브라우저 분포. 기술 스택: Streamlit + pandas + plotly. 깔끔한 UI · 반응형 · 주석 포함."

CATEGORY 04 / 04



AI 앱

Claude API · ChatGPT 활용 지능형 앱

6

PROJECTS

PROJECTS IN THIS CATEGORY

1

고급 · 60분
AI 챗봇 풀스택

Claude API + Supabase로 대화 이력이 저장되는 AI 챗봇 웹앱. React로 구현.

React · Supabase · Claude

2

중급 · 30분
Streamlit AI 챗봇

Claude API 연동 채팅 앱. 대화 이력 유지 + 전력IT 전문가 시스템 프롬프트.

Streamlit · anthropic

3

고급 · 45분
AI 보고서 생성기

CSV 업로드 → Claude API가 분석 보고서를 자동 작성. PDF 다운로드 지원.

Streamlit · pandas · Claude

4

중급 · 25분
AI 문서 요약기

긴 문서·텍스트를 붙여넣으면 Claude가 핵심 요약·키워드 추출·3줄 요약 생성.

Streamlit · anthropic

5

초급 · 20분
AI 번역기

한국어 ↔ 영어 ↔ 일본어 ↔ 중국어 다국어 번역. Claude로 자연스러운 번역 + 문체 선택.

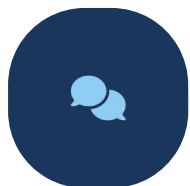
Streamlit · anthropic

6

고급 · 35분
AI 코드 리뷰어

코드 붙여넣기 → Claude가 버그·보안 취약점·개선 사항 분석 + 리팩토링 제안.

Streamlit · anthropic



PROJECT 01 · AI 앱

AI 챗봇 풀스택

Claude API + Supabase로 대화 이력이 저장되는 AI 챗봇 웹앱. React로 구현.

🔧 WHAT YOU'LL BUILD · 만들 것

1 React 챗 UI + 메시지 스트림

2 Claude API 호출

3 Supabase로 대화 이력 저장

📦 TECH STACK

React

Supabase

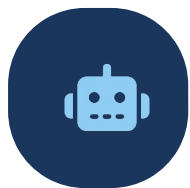
Claude

💡 핵심 팁

21시간의 모든 것이 합쳐지는 종합 결과물
— 친구에게 자랑할 만한 완성도.

📄 STARTER PROMPT · Cursor에 그대로 붙여넣기

"AI 챗봇 풀스택 만들어줘. 기능: React 챗 UI + 메시지 스트림 · Claude API 호출 · Supabase로 대화 이력 저장. 기술 스택: React + Supabase + Claude. 깔끔한 UI · 반응형 · 주석 포함."



PROJECT 02 · AI 앱

Streamlit AI 챗봇

Claude API 연동 채팅 앱. 대화 이력 유지 + 전력IT 전문가 시스템 프롬프트.

🔧 WHAT YOU'LL BUILD · 만들 것

1 `st.chat_input + chat_message`

2 전력IT 전문가 시스템 프롬프트

3 `session_state`로 이력 유지

📦 TECH STACK

Streamlit

anthropic

💡 핵심 팁

Day 3 Part 3 실습
— 한 시간에 만드는 사내 AI 어시스턴트.

📄 STARTER PROMPT · Cursor에 그대로 붙여넣기

"Streamlit AI 챗봇 만들어줘. 기능: `st.chat_input + chat_message` · 전력IT 전문가 시스템 프롬프트 · `session_state`로 이력 유지. 기술 스택: Streamlit + anthropic. 깔끔한 UI · 반응형 · 주석 포함."



PROJECT 03 · AI 앱

AI 보고서 생성기

CSV 업로드 → Claude API가 분석 보고서를 자동 작성. PDF 다운로드 지원.

🔧 WHAT YOU'LL BUILD · 만들 것

- 1 CSV 업로드 + 자동 분석
- 2 Claude로 인사이트 텍스트 생성
- 3 Markdown · PDF 출력

📦 TECH STACK

Streamlit

pandas

Claude

💡 핵심 팁

데이터 + AI = 매주 반복하던 보고서 작업을 버튼 한 번으로.

✍️ STARTER PROMPT · Cursor에 그대로 붙여넣기

"AI 보고서 생성기 만들어줘. 기능: CSV 업로드 + 자동 분석 · Claude로 인사이트 텍스트 생성 · Markdown · PDF 출력. 기술 스택: Streamlit + pandas + Claude. 깔끔한 UI · 반응형 · 주석 포함."



PROJECT 04 · AI 앱

AI 문서 요약기

긴 문서·텍스트를 붙여넣으면 Claude가 핵심 요약·키워드 추출·3줄 요약 생성.

🔧 WHAT YOU'LL BUILD · 만들 것

- 1 긴 텍스트 입력
- 2 Claude 요약 + 키워드 추출
- 3 3줄 / 5줄 / 한 줄 선택

📦 TECH STACK

Streamlit

anthropic

💡 핵심 팁

회의록 · 논문 · 긴 메일
— 시간을 잡아먹던 모든 텍스트를 정리합니다.

🔗 STARTER PROMPT · Cursor에 그대로 붙여넣기

"AI 문서 요약기 만들어줘. 기능: 긴 텍스트 입력 · Claude 요약 + 키워드 추출 · 3줄 / 5줄 / 한 줄 선택. 기술 스택: Streamlit + anthropic. 깔끔한 UI · 반응형 · 주석 포함."



PROJECT 05 · AI 앱

AI 번역기

한국어 ↔ 영어 ↔ 일본어 ↔ 중국어 다국어 번역. Claude로 자연스러운 번역 + 문체 선택.

🔧 WHAT YOU'LL BUILD · 만들 것

- 1 언어 4개 선택 드롭다운
- 2 Claude로 번역 + 문체 옵션
- 3 원문 ↔ 번역 좌우 분할

📦 TECH STACK

Streamlit

anthropic

💡 핵심 팁

Google 번역과 비교
— 문맥이 얼마나 반영되는지 직접 확인합니다.

🔗 STARTER PROMPT · Cursor에 그대로 붙여넣기

"AI 번역기 만들어줘. 기능: 언어 4개 선택 드롭다운 · Claude로 번역 + 문체 옵션 · 원문 ↔ 번역 좌우 분할. 기술 스택: Streamlit + anthropic. 깔끔한 UI · 반응형 · 주식 포함."



PROJECT 06 · AI 앱

AI 코드 리뷰어

코드 붙여넣기 → Claude가 버그·보안 취약점·개선 사항 분석 + 리팩토링 제안.

🔧 WHAT YOU'LL BUILD · 만들 것

- 1 코드 붙여넣기 + 언어 자동 감지
- 2 버그·보안·스타일 분석
- 3 리팩토링 제안 코드 제시

📦 TECH STACK

Streamlit

anthropic

💡 핵심 팁

내가 쓴 코드를 점검받는 도구
— 언제든지 쓸 수 있는 페어 프로그래머.

🔗 STARTER PROMPT · Cursor에 그대로 붙여넣기

"AI 코드 리뷰어 만들어줘. 기능: 코드 붙여넣기 + 언어 자동 감지 · 버그·보안·스타일 분석 · 리팩토링 제안 코드 제시. 기술 스택: Streamlit + anthropic. 깔끔한 UI · 반응형 · 주석 포함."

21시간을 마치며 — 다 챙겼나요?

3일 동안 익힌 핵심 21가지 — 본인 체크해 보세요.

DAY 1 · 프론트엔드

7가지 핵심

- ✓ Cursor IDE 설치 + 단축키 (⌘K · ⌘L · ⌘I)
- ✓ Node · Git · GitHub 환경 구성
- ✓ Vite + React + TS 프로젝트 생성
- ✓ AI에게 요구사항 정의서 받기
- ✓ Component · Props · State 이해
- ✓ React Router 페이지 분리
- ✓ Todo 앱 동작 + Git 첫 커밋

DAY 2 · 백엔드

7가지 핵심

- ✓ REST API 설계 (GET/POST/PUT/DELETE)
- ✓ Express + 미들웨어 구성
- ✓ Supabase 프로젝트 + 테이블 생성
- ✓ RLS 정책으로 본인 데이터만 접근
- ✓ 이메일 회원가입/로그인 구현
- ✓ Google/Kakao OAuth 연동
- ✓ AuthGuard로 보호된 라우트

DAY 3 · 데이터 + AI

7가지 핵심

- ✓ Streamlit 설치 + 첫 앱 실행
- ✓ Python 핵심 6요소 이해
- ✓ 위젯 8개 + 레이아웃 사용
- ✓ CSV 업로드 + pandas 분석
- ✓ 전력 데이터 대시보드 완성
- ✓ Claude API 연동 챗봇
- ✓ Streamlit Cloud 무료 배포 + URL 공유

감사합니다

바이브코딩은 끝이 아니라 시작입니다.

3일 동안 익힌 것을 — 내일 업무부터 적용해 보세요.
가장 작은 자동화 한 개부터 시작하세요. 그게 누적되면 1년 뒤 큰 차이가 됩니다.

총 분량

21시간 · 3일

프로젝트 24선 카탈로그

웹 + Streamlit + AI 풀스택

대상

회사 임직원

비개발자 포함 · 대면 실습

전력 데이터 예제 포함

강의명

Claude Code와 함께하는

풀스택 바이브코딩 3일 워크숍

다음 만남에서도 — Keep coding!

CREDITS · CONTACT

출처 · 문의

이애본 (Aebon Lee, Ph.D) — 드림아이티비즈 대표 · 한신대학교 AI·SW대학 겸임교수

useful-ai.dreamitbiz.com — 강의안·영상 강의 라이브러리

aebon@dreamitbiz.com · 카카오톡 ID aebon

© 2026 이애본 · 발행 홍문원(洪文院) · CC BY-NC-ND 4.0